

The Future Unemployed

THE NEXT WAVE IN TECHNOLOGY

Date, 2026

Gracie Mitchell, Graham McCabe, Reid Holloway
Panther Cloud Air | Panthercloudair@gmail.com

Table of Contents

Architecture **5**
Tier Descriptions 6
Major Algorithms **7**
App.py - Flask Rest API Backend **13**
1. Data Components: 13
2. Module Signatures: 15
3. Connections: 20
4. OMT Models 21
Db.py - Database Connection Layer **23**
1. Data Components 23
2. Module Signatures: 24
simulate.py - 14-Day Simulation Engine **24**
1. Data Components 24
2. Module Signatures: 28
panther_cloud_air_shared.js - Shared Browser Utilities **44**
1. Data Components 44
2. Module Signatures: 47
book.html - **58**
Data Components 58
Module Signatures: 60
Admin.html - admin dashboard **65**
1. Data Components 65
2. Module Signatures 68
Work Breakdown **87**

Table of Figures

Figure 1: Team Logo..... 5
Figure 2: Architecture Diagram..... 6

Table of Tables

Table 1: Table of Components app.py 16
Table 2: Add_cors_headers 16
Table 3: index 17
Table 4: serve_static 17
Table 5: Read-Only Data Endpoints 18
Table 6: api_login 19
Table 7: _make_ref() 19
Table 8: api_create_booking 20
Table 9: api_get_booking(ref) 21
Table 10: Table of Components db.py 24
Table 11: get_connection() 25
Table 12: get_db() 30
Table 13: qall() 30
Table 14: qone() 31
Table 15: qexec() 32
Table 16: bearing() 33
Table 17: jet_stream_factor() 34
Table 18: load_airports() 34
Table 19: load_aircraft() 35
Table 20: load_pricing() 36
Table 21: calc_fare() 36
Table 22: build_timetable_from_static() 37
Table 23: compute_demand() 38
Table 24: build_flight_index() 39
Table 25: _bfs() 40
Table 26: route_passengers() 41
Table 27: apply_challenges() 42
Table 28: simulate_day() 43
Table 29: generate_final_report() 44
Table 30: main() 45
Table 31: showBookingStep() 48
Table 32: navigateTo() 49
Table 33: switchTab() 50
Table 34: checkAuth() 50
Table 35: initNavBar() 51
Table 36: _saveSessionAndRedirect() 52
Table 37: logout() 52
Table 38: applyTheme() 53
Table 39: setTheme() 53
Table 40: getCentsPerNm() 54
Table 41: calcBaseFare() 54

Table 42: areTooClose() 55
Table 43: fillRouteDropdowns() 55
Table 44: showModal() / hideModal() 56
Table 45: showAircraftPanel() 57
Table 46: hideAircraftPanel() 57
Table 47: timeToMinutes() 58
Table 48: fetchAirports() 58
Table 49: fetchAircrafts() 59
Table 50: fetchPricing() 59
Table 51: searchFlights() 61
Table 52: dfs() 62
Table 53: displaySearchResults() 62
Table 55: renderCabinStep() 63
Table 56: renderConfirmStep() 64
Table 57: submitBooking() 64
Table 58: getCpmFare() 70
Table 59: computeCpmAvgFareperPax() 71
Table 60: adjCpmSlider() 71
Table 61: resetCpm() 72
Table 62: computeMarginScale() 72
Table 63: adjSlider() 73
Table 64: updatePricingImpact() 74
Table 65: saveMargins() 74
Table 66: resetMargins() 75
Table 67: applyAutoRoutes() 75
Table 68: buildFleet() 76
Table 69: buildFleetModelStatus() 77
Table 70: buildRouteTables() 77
Table 71: buildPricingAircraftTable() 78
Table 72: addMinsToTime() 79
Table 73: mkRng() 79
Table 74: simRunDay () 80
Table 75: simRunAll() 81
Table 76: simReset() 81
Table 77: fmtM() 82
Table 78: fmtK() 82
Table 79: refreshSimUI() 83
Table 80: buildSparkbar() 84
Table 81: _buildFlightIndx() 85
Table 82: showDayDetail() 85
Table 83: hideDayDetail() 86
Table 84: filterDayDetail() 86
Table 85: renderDayDetail() 87

Table 86: init()..... 88



Figure 1: Team Logo

Architecture

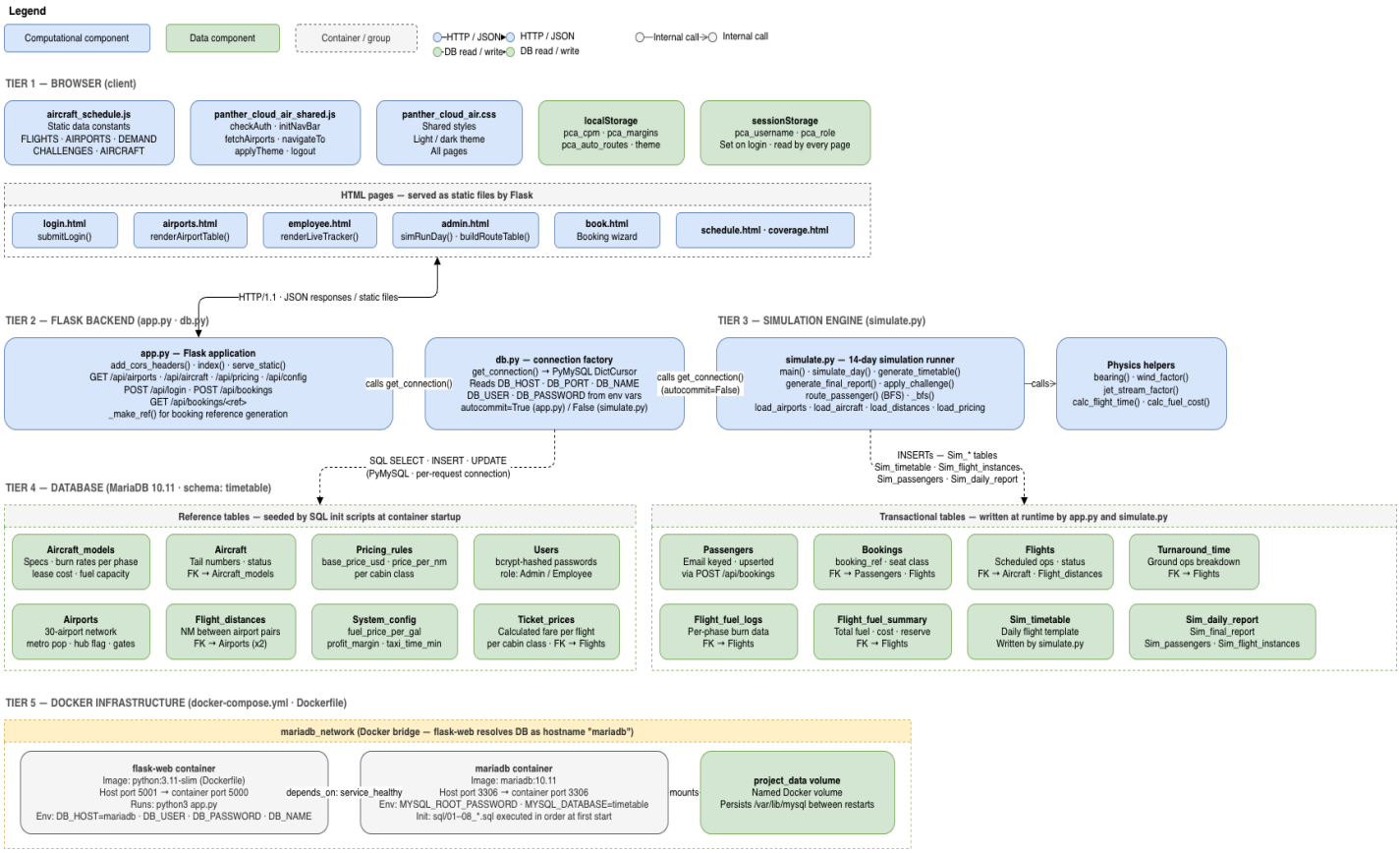


Figure 2: Architecture Diagram

Tier Descriptions

Frontend: contains all client-side components. The six HTML pages (login, airports, employee, admin, book, and schedule/coverage) are served as static files by Flask and each load aircraft_schedule.js (static data constants), panther_cloud_air_shared.js (auth guards, nav helpers, API fetch wrappers), and panther_cloud_air.css (shared styles and theming). sessionStorage holds the logged-in user's role for the duration of the session, while localStorage persists admin pricing settings across sessions.

Flask Backend: consists of two Python files. `app.py` registers all API routes and serves static files, exposing endpoints for airports, aircraft, pricing, config, authentication, and passenger bookings. `db.py` is a connection factory — get_connection() reads database credentials from environment variables and returns a PyMySQL DictCursor connection. Every API request opens a fresh connection and closes it in a finally block; there is no connection pooling.

Simulation Engine: is simulate.py, a standalone command-line script run independently of Flask. It builds a repeating 14-day flight timetable, applies daily operational challenges (weather delays, cancellations, aircraft groundings), routes passengers through the network using a BFS algorithm, and calculates fuel costs using physics helper functions defined within the same file. Results are written to the Sim_* tables in the database.

Database: is a MariaDB instance running the timetable schema. Reference tables (Aircraft, Airports, Pricing_rules, Users, etc.) are seeded by SQL initialization scripts at container startup. Transactional tables (Passengers, Bookings, Flights, fuel logs, and the Sim_* family) are written at runtime by app.py and simulate.py respectively.

Docker Infrastructure: wraps both services in an isolated bridge network. The flask-web container runs the Flask app on port 5001 and waits for the mariadb container to pass its health check before starting. A named volume, project_data, persists the database files across container restarts.

Major Algorithms

1. Password Authentication (api_login)

```
1. Strip whitespace from username + password
2. IF either is empty → return HTTP 400
3. SELECT user row WHERE username = ? FROM Users
4. IF row exists AND check_password_hash(row.password, password)
       → return HTTP 200 { ok, username, user_type }
5. ELSE → return HTTP 401
6. FINALLY: close connection
```

2. Booking Reference Generation (_make_ref)

```
ALPHABET ← [A-Z] + [0-9]  (36 chars)
suffix   ← 5 random chars chosen from ALPHABET
RETURN 'PCA-' + suffix                  // e.g. PCA-A3X7K
```

3. Passenger Upsert (api_create_booking)

```
1. FOR each required field: IF missing → return HTTP 400 with field name
2. dob ← body.get('dob', '2000-01-01')
3. ref ← _make_ref()
4. EXECUTE:
       INSERT INTO Passengers (first_name, last_name, email, date_of_birth) VALUES (...)
       ON DUPLICATE KEY UPDATE
       first_name=VALUES(...), last_name=VALUES(...), date_of_birth=VALUES(...)
5. RETURN HTTP 200 { ok: true, booking_ref: ref }
6. FINALLY: close connection
```

4. Static File Security Filter (serve_static)

```
IF '/' IN filename                          → return 404
IF extension NOT IN [.html, .js, .css]  → return 404
RETURN send_from_directory('.', filename)
```

5. Gravity-Model Demand Computation (compute_demand)

This algorithm estimates how many passengers fly between each city pair per day. It is modeled after the real-world gravity model used in transportation planning — the idea being that larger cities generate more travelers, and travelers are more likely to fly toward larger destinations. The total network population is calculated first, then for every origin-destination pair the formula scales the origin city's daily flyers by the destination's share of the remaining network population. Any pair producing less than 0.5 passengers per day is dropped as negligible. The result is a dictionary that drives all passenger generation in the 14-day simulation.

```
total_network_pop ← SUM of all airport metro populations
FOR each origin airport:
  origin_pop          ← airports[origin].metro_pop
  other_metros_pop    ← total_network_pop − origin_pop
  daily_pax_from_origin ← origin_pop × DAILY_FLY_RATE × MARKET_SHARE

  FOR each destination airport (dest ≠ origin):
    dest_pop        ← airports[dest].metro_pop
    daily_passengers ← daily_pax_from_origin × (dest_pop / other_metros_pop)
      IF daily_passengers ≥ 0.5:
        demand[(origin, dest)] ← daily_passengers
RETURN demand
```

6. Distance-Based Fare Calculation (calc_fare / calcBaseFare)

This algorithm computes the ticket price for a given itinerary. Two implementations exist — one in Python for the simulation and one in JavaScript for the browser booking wizard — but both use the same linear formula: a base price plus a per-nautical-mile rate, multiplied by a margin percentage. The Python version uses exact distances from the database. The JavaScript version approximates distance by multiplying total block minutes by 6, which reflects the rule of thumb that a typical jet covers roughly 6 nautical miles per minute. A $49 floor prevents absurdly cheap fares on very short routes.

Server-side (simulate.py):

```
FUNCTION calc_fare(seat_class, dist_nm, pricing):
  rule ← pricing[seat_class]
  RETURN ROUND((rule.base + rule.per_nm × dist_nm) × (1 + rule.margin / 100), 2)
```

Client-side (shared.js):

```
FUNCTION calcBasefare(legs):
  total_block_minutes ← 0
  FOR each leg:
        IF arr_minutes > dep_minutes:
            total_block_minutes += arr_minutes - dep_minutes
        ELSE:
            Total_block_minutes += (1440 - dep_minutes) + arr_minutes
    estimated_nm ← total_block_minutes x 6
    RETURN MAX(49, ROUND(estimated_nm x getCentsPerNm() / 100))
```

7. BFS Passenger Routing (_bfs / route_passenger)

This is the most complex algorithm in the project. For each simulated passenger, the system needs to find a valid sequence of flights from their origin to their destination using only flights that have not been cancelled and still have seats available. Breadth-First Search was chosen because it naturally finds the path with the fewest legs — so a nonstop is always preferred over a one-stop, and a one-stop is always preferred over a two-stop. The graph nodes are airports and the edges are available flight instances. Each BFS state tracks the current airport, the earliest minute the passenger is free to depart, and the list of legs taken so far. A visited dictionary keyed on airport and a coarse 90-minute time bucket prevents the search from revisiting equivalent states. Hub airports are prioritized as connection points before non-hub airports, which reflects the real hub-and-spoke structure of the network.

route_passenger wraps _bfs with a progressive fallback: it first attempts routing with a 30-minute minimum connection time, then retries with 15 minutes, and finally tries with no minimum connection from the earliest possible departure of the day. This models real airline irregular operations — if a passenger cannot make a standard connection they are not immediately stranded, but instead the system tries progressively looser constraints before giving up.

8. Operational Challenge Engine (apply_challenge)

Each of the 14 simulation days has a different operational disruption. This algorithm takes the day's challenge parameters and applies them as a series of filters over every flight instance for that day. Random cancellations and delays are applied probabilistically using a random number compared against the challenge's cancel and delay percentages. Delay amounts are drawn from a normal distribution so they vary realistically rather than being uniform. The jet stream challenge uses the bearing between origin

and destination coordinates to determine direction of travel — eastbound flights get a time reduction and westbound flights get a time penalty. The western cancellation challenge checks whether either endpoint of a flight falls in the western airport set. The AOG (Aircraft on Ground) challenge finds the first aircraft departing from the affected hub and cancels every flight that was scheduled to operate that day.

9. Route Aggregation and Load Factor Ranking (renderBusiestRoutes)

This algorithm identifies the six busiest routes operating on a given day and flags any that are underperforming. It performs an in-memory group-by over the full FLIGHTS array, using the origin-destination pair as the key. For each group it accumulates total passengers, total seats, and flight count. Once all flights are grouped, the routes are sorted by total passengers in descending order and the top six are taken. Load factor — the percentage of seats filled — is then computed per route. Any route below 30% capacity is flagged with a warning badge to alert staff that the route may need a promotion or schedule adjustment.

10. Proximity Exclusion Check (areTooClose)

This algorithm enforces the rule that airports within 150 miles of each other have no direct service. Rather than computing distances at runtime, the excluded pairs are pre-defined as a Set of canonical sorted strings in `aircraft_schedule.js`. The lookup is O(1). The function checks both orderings of the pair since the Set stores them in alphabetical order but callers may pass the airports in either order. This check is used in the booking wizard to block passengers from searching same-metro routes, and in the BFS routing to skip legs that would violate the rule during connection building.

```
TOO_CLOSE ← Set { "BWI-DCA", "EWR-JFK", "LAX-SAN", ... } // 17 pairs

FUNCTION areTooClose(airportA, airportB):

    RETURN (airportA + '-' + airportB) IN TOO_CLOSE

      OR (airportB + '-' + airportA) IN TOO_CLOSE
```

11. DFS Flight Search (searchFlights)

This algorithm finds available itineraries between an origin and destination by running a Depth-First Search over an adjacency graph built from the FLIGHTS array. Each node in the graph is an airport and each edge is a scheduled flight. The search is called recursively, building up a path of legs one flight at a time. Before adding any flight to the path, four constraints are checked: the flight number must not already appear in the current path (no flight reuse), the origin-destination pair must not violate the 150-mile proximity rule, the connection gap must be at least 30 minutes and no more than 360 minutes, and recursion only continues if the remaining stop budget allows it. When a flight's destination matches the target airport, the complete path is saved as a found itinerary. The search stops entirely once 6 itineraries have been found to keep the interface responsive.

```
FUNCTION searchFlights():
  IF origin = destination: alert and RETURN
  IF areTooClose(origin, destination): show rule message and RETURN
  adjacencyGraph ← {}
  FOR each flight in FLIGHTS:
    adjacencyGraph[flight.from].append(flight)
  foundItineraries ← []
  FUNCTION dfs(currentAirport, pathSoFar):
    IF foundItineraries.length ≥ 6: RETURN
    FOR each flight departing currentAirport:
      IF flight.num already in pathSoFar: CONTINUE        // no flight reuse
      IF areTooClose(flight.from, flight.to): CONTINUE    // 150-mile rule
      IF pathSoFar not empty:
        gap ← timeToMinutes(flight.dep) − timeToMinutes(lastLeg.arr)
        IF gap < 0: gap += 1440                           // midnight wrap
        IF gap < 30 OR gap > 360: CONTINUE                // connection window
      updatedPath ← pathSoFar + [flight]
      IF flight.to = destination:
```

```
            foundItineraries.append(updatedPath)
        ELSE IF pathSoFar.length < maxStops:
            dfs(flight.to, updatedPath)                    // recurse with stop budget
    dfs(origin, [])
    displaySearchResults()
```

12 Fare Calculation with Margin Multiplier (getMarginMultiplier / calcLegFare / renderConfirmStep)

The booking wizard uses a three-layer fare formula. The first layer estimates distance from block time. The second applies the admin-configured cents-per-nautical-mile rate. The third applies a margin multiplier that reflects the current profit margin settings set by an admin. The margin multiplier is computed as the ratio of the current average margin to the baseline average of 24.5%, so at default settings it equals exactly 1.0 and has no effect on fares. A cabin class multiplier is then applied on top — Economy is 1× and First Class is 4×. Finally, taxes and fees are added: 11.5% federal excise and 2% airport surcharge.

```
FUNCTION getMarginMultiplier():
    margins ← read pca_margins from localStorage (default: sh=18, mh=22, lh=26, intl=32)
    BASE_AVG ← (18 + 22 + 26 + 32) / 4     // = 24.5%
    curr_avg ← (margins.sh + margins.mh + margins.lh + margins.intl) / 4
    RETURN (1 + curr_avg / 100) / (1 + BASE_AVG / 100)
FUNCTION calcLegFare(leg):
    block_minutes ← timeToMinutes(leg.arr) − timeToMinutes(leg.dep)
    IF block_minutes ≤ 0: block_minutes += 1440       // overnight leg
    RETURN MAX(49, ROUND(block_minutes × 6 × getCentsPerNm() / 100 × getMarginMultiplier()))
FUNCTION renderConfirmStep():
    mult  ← { e: 1, f: 4 }[selectedClass]
    base  ← SUM of calcLegFare(leg) for each leg in selectedLegs
    fare  ← ROUND(base × mult)
```

```
tax   ← ROUND(fare × 0.115)
sur   ← ROUND(fare × 0.02)
total ← fare + tax + sur
window._total ← total
RENDER price breakdown showing base, taxes, surcharge, and total
```

App.py - Flask Rest API Backend

1. Data Components:

Name	Purpose	Type
Aircraft_models	Specs per aircraft type: seats, range, cruise speed, fuel capacity, burn rates per flight phase (taxi / takeoff / ascent / cruise / descent / landing), monthly lease cost. Referenced by Aircraft.	MariaDB Table
Aircraft	Individual tail numbers with status (active / maintenance / retired) and total flight hours. FK → Aircraft_models.	MariaDB Table
Airports	30-airport PCA network: IATA code, city, state, timezone, metro population, hub flag (4 hubs / 26 spokes), gates available.	MariaDB Table
Flight_distances	Point-to-point distances in nautical miles between all airport pairs. Both directions stored as separate rows. is_active flag disables a route without deleting the historical record. FK → Airports (origin_id, destination_id).	MariaDB Table
Flights	Scheduled and completed flights: flight number, linked distance and aircraft, scheduled/actual departure and arrival times, status (scheduled / boarding / airborne / landed / cancelled /	MariaDB Table

	diverted).	
Turnaround_time	Ground time breakdown per flight: disembark, clean/crew change, board, door close, optional refuel (+10 min). Minimum ground time: 55 min (no refuel) / 65 min (with refuel). FK → Flights.	MariaDB Table
Flight_fuel_logs	Per-phase fuel burn across 9 phases (taxi_out, takeoff, ascent_low, ascent_high, cruise, descent_high, descent_low, landing, taxi_in). Stores duration_min and fuel_burned_gal. FK → Flights.	MariaDB Table
Flight_fuel_summary	Rolled-up fuel totals per flight: total burned, fuel cost USD, reserve fuel (10% FAA requirement), total loaded. FK → Flights.	MariaDB Table
Pricing_rules	Admin-defined fare parameters per cabin class (economy / business / first): base_price_usd and price_per_nm.	MariaDB Table
Ticket_prices	Calculated ticket price per flight per cabin class, derived at runtime from Pricing_rules + distance + profit_margin from System_config. FK → Flights.	MariaDB Table
System_config	Key-value simulation settings: profit_margin, fuel_price_per_gal, taxi_time_min.	MariaDB Table
Passengers	Personal info per passenger keyed on email. Upserted on each booking via api_create_booking().	MariaDB Table
Bookings	Flight reservations: booking_ref, passenger, flight, seat class, seat number, price paid, status (confirmed / cancelled / completed). FK → Passengers, FK → Flights. Note: booking_ref is generated in Python but NOT persisted here.	MariaDB Table
Users	Staff credentials: username, bcrypt-hashed password, role (Admin / Employee).	MariaDB Table
booking_ref	Randomly generated PCA-XXXXX reference returned by api_create_booking(). Ephemeral — not persisted to the DB.	Python str (per-request)

Flask app	Central application object. Owns all routes and the CORS after_request hook.	flask.Flask singleton

Table 1: Table of Components app.py

2. Module Signatures:

2.1 Infrastructure & Static File Serving:

Add_cors_headers

Function	add_cors_headers	Kind	Function (after_requesthook)
Signature	Def add_cors_headers(response)	Returns	flask.Response with CORS headers injected
Description	Attaches Access-Control-Allow-* headers to every HTTP response so the front-end can call the API from any origin (localhost, Docker, or file://).		
Preconditions	Flask must have completed routing and produced an initial response object.		
Exceptions	No explicit exceptions		
Postconditions	Allow-Origin → '*', Allow-Headers → 'Content-Type', Allow-Methods → 'GET, POST, OPTIONS' set on every response. No DB changes.		

Table 2: Add_cors_headers

Parameters

Name	Purpose	Type	Mode
Responses	Flask response object to be sent	flask.Response	in/out

index()

Function	index	Kind	Function (GET)
Signature	def index() → Response	Returns	flask.Response — HTTP 200 with login.html, or 404
Description	Serves login.html at the application root. Entry point for all portal users		
Preconditions	login.html must exist in the project root directory.		
Exceptions	404 Not Found if login.html is missing (raised by Werkzeug).		
Postconditions	Returns login.html with HTTP 200. No DB access. No state changes.		

Table 3: index

serve_static

Function	service _static	Kind	Function (GET path:filename)
Signature	def serve_static(filename: str) → Response	Returns	flask.Response — HTTP 200 with file, or ('Not found', 404)
Description	Serves any .html, .js, or .css file from the project root. Blocks filenames containing '/' to prevent directory traversal.		
Preconditions	filename contains no '/' and ends with .html, .js, or .css. File must exist on disk.		
Exceptions	('Not found', 404) if filename has '/' or a disallowed extension. Werkzeug 404 if file passes validation but is missing on disk.		
Postconditions	Returns static file with HTTP 200 on success. No DB access.		

Table 4: serve_static

Parameters

Name	Purpose	Type	Mode
filename	URL path segment capturing the requested filename	str (URL path param)	input

2.2 Read-Only Data Endpoints (GET)

All four endpoints share the same pattern: open DB connection → query → convert Decimal to float → close connection → return JSON.

Endpoint	Function	Returns	Preconditions	Exceptions / Post
GET /api/airports	api_airports()	JSON array of airport dicts. is_hub cast to bool.	Airports table exists; DB reachable.	HTTP 500 on DB error. No writes.
GET /api/aircraft	api_aircraft()	JSON array of Aircraft JOIN Aircraft_models. Decimals → float.	Both tables exist; DB reachable.	HTTP 500 on DB error. No writes.
GET /api/pricing	api_pricing()	JSON array of 3 pricing dicts (economy, business, first).	Pricing_rules table exists (may be empty).	Fallback to hard-coded defaults if table empty. No writes.
GET /api/config	api_config()	Flat JSON dict { config_key: float }.	System_config table exists; DB reachable.	HTTP 500 on DB error. Empty {} if no rows. No writes.

Table 5: Read-Only Data Endpoints

2.3 Authentication & Booking (POST)

api_login()

Function	api_login	Kind	Function (POST endpoint)
Signature	def api_login() → Response [POST /api/login]	Returns	HTTP 200 / 400 / 401 JSON
Description	Authenticates a staff user. Look up the username in the Users table and compare the supplied password against the bcrypt hash using check_password_hash.		

Preconditions	Requests body must be valid JSON with non-empty username and password fields.
Exceptions	HTTP 400: missing/empty username or password. HTTP 401: username not found or password mismatch. HTTP 500: DB connection failure (unhandled).
Postconditions	HTTP 200 { ok: true, username, user_type } on success. user_type drives navigation: Admin → admin.html, Employee → employee.html. No DB writes. Connection closed in finally.

Table 6: api_login

Parameters

Name	Purpose	Type	Mode
username	Staff username from JSON request body	str	Input
password	Plaintext password from JSON request body	str	Input

_make_ref()

Function	_make_ref	Kind	Function (pure utility)
Signature	def _make_ref() → str	Returns	str — 'PCA-XXXXX' (9 characters, ~60.5M combinations)
Description	Generates a random booking reference using 5 characters from [A-Z0-9]. Internal helper called by api_create_booking.		
Preconditions	None.		
Exceptions	None.		
Postconditions	Returns a string matching PCA-[A-Z0-9]{5}. No side effects, no DB access.		

Table 7: _make_ref()

api_create_booking()

Function	api_create_booking	Kind	Function (POST endpoint, DB write)
Signature	def api_create_booking() → Response [POST /api/bookings]	Returns	HTTP 200 / 400 JSON

Description	Validates booking fields, generates a reference via _make_ref(), then upserts a row in the Passengers table (INSERT … ON DUPLICATE KEY UPDATE on email). flight_id FK is not populated as scheduled flights live in the static JS array.
Preconditions	JSON body must contain: first_name, last_name, email, seat_class, total_price_usd. dob is optional (defaults to 2000-01-01).
Exceptions	HTTP 400 { ok: false, error: 'Missing field: <name>' } for any absent required field. HTTP 500 on DB connection or SQL error (unhandled).
Postconditions	HTTP 200 { ok: true, booking_ref: 'PCA-XXXXX' }. Passengers row inserted or updated. booking_ref is NOT persisted to DB. Connection closed in finally.

Table 8: api_create_booking

Parameters

Name	Purpose	Type	Mode
first_name	Passenger first name	str (JSON body)	Input
last_name	Passenger last name	str (JSON body)	Input
email	Email address; unique key for upsert	str (JSON body)	Input
seat_class	Cabin class: economy / business / first	str (JSON body)	Input
total_price_usd	Total fare in USD	float (JSON body)	Input
dob	Date of birth (optional, default 2000-01-01)	str ISO date	Input

api_get_booking(ref)

Function	api_get_booking	Kind	Function (GET endpoint)
Signature	def api_get_booking(ref: str) → Response [GET /api/bookings/<ref>]	Returns	HTTP 200 JSON
Description	Echoes the booking reference in uppercase for the confirmation screen. Decorative — no DB access.		
Preconditions	ref is a non-empty URL path segment.		
Exceptions	None		

Postconditions	• Returns { ok: true, booking_ref: ref.upper() }. No DB access, no state changes.

Table 9: api_get_booking(ref)

Parameters

Name	Purpose	Type	Mode
ref	Booking reference captured from the URL path	str (URL path param)	Input

3. Connections:

Name	Purpose	Data Type	Protocol / Sync	Filtering
Client ↔ Flask HTTP	Carries all browser requests and responses (JSON for API, HTML/JS/CSS for static).	HTTP/1.1 over TCP	Stateless req/response. No session cookies — state in localStorage. Single-threaded in debug mode.	CORS headers on all responses. Static handler blocks '/' and non-.html/.js/.css extensions.
Flask ↔ MariaDB	Executes parameterised SQL queries; returns rows as Python dicts via DictCursor.	MySQL wire protocol	New connection per request (no pooling). Synchronous / blocking. Closed in finally.	Parameterised %s placeholders prevent SQL injection. Decimal fields cast to float before JSON.
Flask ↔ File System	Reads static assets from project root and streams them to the browser.	POSIX file I/O	Synchronous read. No caching headers set.	serve_static validates extension + absence of '/' before delegating to send_from_directory.

Table 9: Connections Table app.py

4. OMT Models

5.1 Static Object Model

Class / Entity	Key Responsibilities	Relationships
Flask (app)	Registers all routes; fires CORS after_request hook.	Owns all route handlers (composition)
Users	Staff credentials; used only by api_login.	Queried by Flask on POST /api/login
Aircraft	Individual tail numbers + operational status.	Many-to-1 → Aircraft_models (model_id FK)
Aircraft_models	Shared specs: seats, speed, range, burn rates per phase, lease cost.	1-to-many ← Aircraft
Airports	Network nodes; metro pop drives display order.	Referenced by Flight_distances (origin_id, destination_id FK)
Flight_distances	Point-to-point distances between airport pairs.	Many-to-1 → Airports (×2); 1-to-many ← Flights
Flights	Core flight records; links aircraft, route, and schedule.	FK → Aircraft, FK → Flight_distances; referenced by Turnaround_time, fuel tables, Bookings, Ticket_prices
Turnaround_time	Ground time breakdown per flight.	1-to-1 → Flights
Flight_fuel_logs	Per-phase fuel burn (9 phases) per flight.	Many-to-1 → Flights
Flight_fuel_summary	Rolled-up fuel totals per flight.	1-to-1 → Flights
Pricing_rules	One row per cabin class (economy / business / first).	Referenced by Ticket_prices at runtime
Ticket_prices	Calculated fare per flight per cabin class.	Many-to-1 → Flights
System_config	Simulation-wide key-value settings.	Read by api_config(); used by simulate.py for fuel pricing

Passengers	Unique per email; upserted on each booking.	1-to-many ← Bookings; written by api_create_booking()
Bookings	Reservations linking passengers to flights. Not yet written to by app.py.	FK → Passengers, FK → Flights

Table 10: Static Object Model app.py

5.2 State Charts — Request Lifecycles (POST Endpoints)

Phase	api_login	api_create_booking
1 — Parse	Extract username + password from JSON body.	Extract all fields from JSON body; default dob if absent.
2 — Validate	Check both fields non-empty → HTTP 400 if not.	Check each required field present → HTTP 400 with field name if not.
3 — Execute	SELECT user; check_password_hash → HTTP 200 or HTTP 401.	Call _make_ref(); INSERT … ON DUPLICATE KEY UPDATE → HTTP 200.
4 — Cleanup	DB connection closed in finally block.	DB connection closed in finally block.

Table 10: State Charts app.py

5.3 Event Trace Tables

#	From	To	Message
1	Browser	POST /api/login	{ username, password }
2	Flask	MariaDB Users	SELECT WHERE username = ?
3	MariaDB	Flask	User row or empty result
4	Flask	werkzeug	check_password_hash(hash, password)
5	Flask	Browser	HTTP 200 { ok, username, user_type } OR HTTP 401
6	Browser	localStorage + Router	Store user_type; navigate to admin.html or employee.html

Table 11: Login Use Cases

#	From	To	Message
1	Browser	POST /api/bookings	{ first_name, last_name, email, seat_class, total_price_usd, dob? }

2	Flask	Flask (internal)	Validate required fields
3	Flask	_make_ref()	Generate reference code
4	Flask	MariaDB Passengers	INSERT … ON DUPLICATE KEY UPDATE
5	Flask	Browser	HTTP 200 { ok: true, booking_ref: 'PCA-XXXXX' }
6	Browser	Confirmation screen	Display booking reference to passenger

Table 12: Booking Use Cases

Db.py - Database Connection Layer

1. Data Components

Name	Purpose	Type
DB_HOST	Hostname or Docker service name of the MariaDB server	str (env var, default “localhost”)
DB_PORT	TCP port for MariaDB (default 3306)	int (env var)
DB_NAME	Database schema name (’timetable’)	str (env var)
DB_USER	Database login username	str (env var)
DB_PASSWORD	Database login password	str (env var)
PyMySQL Connection	Active connection object returned by ‘get_connection()’ uses ‘DictCursor’ so rows come back as ’{column: value}` dicts	‘pymysql.connections.Connection’

Table 10: Table of Components db.py

2. Module Signatures:

get_connection()

Type: Function **Return Type:** pymysql.connections.Connection

Parameters:

Description	Opens and returns a new PyMySQL connection to the MariaDB timetable schema. Uses DictCursor so all rows are returned as dicts (Column_name → value) and sets autocommit=True so every INSERT/UPDATE in app.py is committed immediately without an explicit conn.commit() call.
Preconditions	MariaDB must be reachable at the host/port specified by environment variables. The timetable schema and the DB_USER account must already exist.
Exceptions	pymysql.OperationalError if the host is unreachable, credentials are wrong, or the schema does not exist.
Postconditions	Returns an open, ready-to-use connection. The caller is responsible for closing it (typically in a finally block).

Table 11: get_connection()

simulate.py - 14-Day Simulation Engine

1. Data Components

Name	Purpose	Type
STATIC_FLIGHTS	Hard-coded list of all 245 scheduled flights mirroring the browser's 'FLIGHTS' array. Fields: 'num', 'tail', 'type', 'frm', 'to', 'dep', 'arr', and 'seats'	Python 'list[dict]' (constant)
CHALENGES	Maps each simulation day (1--14) to a tuple	Python 'dict[int, tuple]' (constant)

	‘(description, params)’ that controls which challenged is distributed based on the day	
COORDS	Approximates the latitude and longitude coordinates for each airport. Used by ‘bearing’ for the jet-stream challenge	Python ‘dict[str, tuple]’ (constant)
HUBS	List of all hub IATA codes which are used to prioritize connections to achieve optimal passenger demand.	Python ‘list[str]’ (constant)
WESTERN	A set of western airport IATA codes to classify routes	Python ‘set[str]’ (constant)
EAST_OF_100W	Set of airports east of 100th meridian for the Day 12 challenge	Python ‘set[str]’ (computed constant)
SIM_START	Calendar date of Day 1 (‘2025-03-01’)	‘datetime.date’ (constant)
FUEL_PRICE	Jet fuel price in SD/gallon used in cost calculations. (‘6.19’)	‘float’ (constant)
LANDING_FEE	Constant landing fee in USD (‘$2000’)	‘float’ (constant)
MARKET_SHARE	Fraction of city demand PCA has (‘0.02’ = 2%)	‘float’ (constant)
DAILY_FLY_RATE	Fraction of a metro population that flies any given day (‘0.005’)	‘float’ (constant)
CLASS_SPLIT	Booking weight per cabin class: ‘{economy: 0.80, business: 0.15, first: 0.05}’	Python ‘dict[str, float]’ (constant)
Airports	Reference table read at startup. Give metro population to calculate demand	MariaDB Table (read)
Aircraft / Aircraft_models	Reference tables showing	MariaDB Tables (read)

	fuel burn rate, leasing costs, seat counts, and tail number mapping by ID	
Flight_distances	Gives distance in nautical miles and foreign key ID for every route flown	MariaDB Table (read)
Pricing_rules	Base price and per-nm rate for every passenger class using ‘calc_fare()’	MariaDB Table (read)
System_config	Reads ‘profit_margin’ to apply correct fares	MariaDB Table (read)
Sim_timetable	Stores the base schedule from ‘STATIC_FLIGHTS’. Rewritten once before the simulation loops	MariaDB Table (write)
Sim_flight_instances	One row per flight per day with scheduled and actual times, status, delay reason, and passenger count	MariaDB Table (write)

Sim_passengers	One row per demand of passengers with origin, destination, seat class, fare, and status of arrived or stranded	MariaDB Table (write)
Sim_passenger_legs	One row per flight leg taken by each passenger. It also links passengers to flight instances	MariaDB Table (write)
Sim_daily_report	Daily metrics such as revenue, fuel cost, landing fees, leasing costs, P&L, flight counts, and delay totals	MariaDB Table (write)
Sim_final_report	Single 14-day summary written after all simulation days are completed	MariaDB Table (write)

get_db	Opens a MariaDB connections with	Python function

	‘autocommit=False’ for atomic day commits	
qall	Fetches all rows using SQL	Python function
qone	Fetches first row using SQL	Python function
qexec	Executes write statement to the database	Python function
bearing	Computes compass bearing between two airports using trigonometry for wind calculations	Python function
jet_stream_factor	Returns a flight’s duration flight-time difference based on wind bearing (headwind or tailwind)	Python function
load_airports	Loads airport reference data from the database into in-memory dicts at startup	Python function
load_aircraft	Loads aircraft reference data from the database into in-memory dicts at startup	Python function
load_distances	Loads airport distance data from the database into in-memory dicts at startup	Python function
load_pricing	Loads pricing data from the database into in-memory dicts at startup	Python function
calc_fare	Calculates ticket price from database into in-memory dicts at startup	Python function
built_timetable_from_static	Converts ‘STATIC_FLIGHTS’ into the function ‘Sim_timetable’ rows and returns the daily	Python function

	timetable	
compute_demand	Applies a model to produce a daily demand on every origin and destination.	Python function
build_flight_index	Builds an index of non-cancelled flights that are grouped by their route for the BFS sorting algorithm	Python function
_bfs	Breadth-First Search algorithm to determine passenger connections after challenges are applies	Python function
route_passenger	A constant refresh for '_bfs' to apply calculated data	Python function
apply_challenge	Adds daily challenges to set flights	Python function
simulate_day	Puts together all the steps of one simulation day and commits its results	Python function
generate_final_report	Summates 14 daily dicts and writes teh 'Sim_final_report' row	Python function
main	Entry point, clears prior sim data, loads reference data, runs 14 days, writes final report	Python function

2. Module Signatures:

get_db()

Type: Function **Return Type:** pymysql.connections.Connection

Parameters: none

Description	Opens and returns a PyMySQL connection identical to 'get_connection()' except 'autocommit=False'. This

	allows ‘simulate_day()’ to batch all inserts for one day into a single commit, so a mid-day failure can be rolled back without partial data in the DB.
Preconditions	MariaDB must be reachable at the configured host/port. The timetable schema must exist.
Exceptions	pymysql.OperationalError if the connection cannot be established.
Postconditions	Returns an open connection with ‘autocommit=False’. The caller must call ‘conn.commit()’ or ‘conn.rollback()’ and then ‘conn.close()’.

Table 12: get_db()

qall()

Type: Function **Return Type:** ‘list[dict]’

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database connection	pymysql.Connection	input
sql	SQL SELECT statement (may contain ‘%s’ placeholders)	str	input
args	Positional bind parameters for the query	tuple	input

Table 13: qall()

Description	Executes the given SQL SELECT using parameterized ‘%s’ placeholders and returns all rows as a list of dicts via ‘DictCursor.fetchall()’ Thin wrapper to avoid repeating cursor boilerplate throughout ‘simulate.py’.

Preconditions	‘conn’ must be open. ‘sql’ must be a valid SELECT statement. The number of ‘%s’ placeholders must match ‘len(args)’.
Exceptions	‘pymysql.ProgrammingError’ on malformed SQL. ‘pymysql.OperationalError’ if the connection drops.
Postconditions	Returns a (possibly empty) list of row dicts. The connection remains open. No writes.

qone()

Type: Function **Return Type:** ‘dict’ or ‘None’

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database connection	pymysql.Connection	input
sql	SQL SELECT statement	str	input
args	Positional bind parameters	tuple	input

Description	Calls ‘qall()’ and returns only the first element, or ‘None’ if the result set is empty. Used when exactly one row is expected.
Preconditions	Same as ‘qall’.
Exceptions	Same as ‘qall’.
Postconditions	Returns the first dict row or ‘None’. No writes.

Table 14: qone()

qexec()

Type: Function **Return Type:** ‘int’

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database connection	pymysql.Connection	input
sql	SQL INSERT / UPDATE / DELETE statement	str	input
args	Positional bind parameters	tuple	input

Description	Executes an INSERT, UPDATE, or DELETE with parameterized placeholders and returns the auto-generated primary key of the last inserted row. Used by ‘simulate_day()’ to insert flight instances and passenger records and immediately capture their IDs for foreign key linking.
Preconditions	‘conn’ must be open. ‘sql’ must be a valid write statement. ‘autocommit’ may be ‘False’. Then the caller controls when to commit.
Exceptions	‘pymysql.IntegrityError’ on constraint violations. ‘pymysql.OperationalError’ on connection failure.
Postconditions	The row is written to the DB (but not necessarily committed). Returns the ‘lastrowid’ integer.

Table 15: qexec()

bearing()

Type: Function **Return Type:** ‘float’ (compass bearing in degrees)

Parameters:

Name	Purpose	Data Type	Mode

orig	IATA code of the origin airport	str	input
dest	IATA code of the destination airport	str	input

Description	Computes the initial compass bearing from ‘orig’ to ‘dest’ using the trigonometry formula: ‘atan2(sin(Δlon)·cos(lat2), cos(lat1)·sin(lat2) – sin(lat1)·cos(lat2)·cos(Δlon))’ which is converted to degrees mod 360. A bearing of 180--360° indicates a westbound flight.
Preconditions	Both ‘orig’ and ‘dest’ must be keys in the ‘COORDS’ dict. Lat/lon values are in decimal degrees.
Exceptions	‘KeyError’ if either IATA code is absent from ‘COORDS’
Postconditions	Returns a float in [0,360). No side effects. No DB access.

Table 16: bearing()

jet_stream_factor()

Type: Function **Return Type:** ‘float’

Parameters:

Name	Purpose	Data Type	Mode
bearing_degrees	Compass bearing of the flight	float	input
eastward_bonus	Fractional speed bonus for eastbound flights	float	input

Description	Returns a duration scale factor driven by the sine of the bearing. Eastbound flights (bearing near 90°) get a tailwind shortening duration; westbound (near 270°) get a headwind lengthening it. The formula is: ‘1.0 – sin(bearing_rad) * eastward_bonus’.

Preconditions	‘bearing_degrees’ must be in [0, 360). ‘eastward_bonus’ should be in (0, 1).
Exceptions	None
Postconditions	Returns a float near 1.0. No side effects. No DB access.

Table 17: jet_stream_factor()

load_airports()

Type: Function **Return Type:** ‘dict[str, dict]”

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database connection	‘pymysql.Connection’	input

Description	Fetches all rows from the ‘Airports’ table and returns them as a dict keyed by ‘iata_code’. Used by ‘compute_demand()’ to access metro populations for every airport without repeated DB queries.
Preconditions	‘Airports’ table must exist and be populated. ‘Conn’ must be open.
Exceptions	‘pymysql.OperationalError’ or DB failure.
Postconditions	Returns a dict with one entry per airport. No writes.

Table 18: load_airports()

load_aircraft()

Type: Function **Return Type:** ‘dict[str, dict]”

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database	‘pymysql.Connecti	input

	connection	on’	

Description	JOINs ‘Aircraft’ with ‘Aircraft_models’ and filters to ‘status = ‘active’’ aircraft only. Returns a dict keyed by ‘tail_number’ so ‘build_timetable_from_static()’can resolve each static flight's tail to a DB ‘aircraft_id’.
Preconditions	‘Aircraft’ and ‘Aircraft_models’ tables must exist. ‘conn’ must be open
Exceptions	‘pymysql.OperationalError’ or DB failure.
Postconditions	Returns a tuple of two dicts. Routes with ‘is_active = FALSE’ are excluded. No writes.

Table 19: load_aircraft()

load_pricing()

Type: Function **Return Type:** ‘dict[str, dict]’’

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database connection	‘pymysql.Connection’	input

Description	Reads ‘Pricing_rules’ and the ‘profit_margin’ entry from ‘System_config’, then assembles a dict mapping each seat class to its pricing parameters. Used by ‘calc_fare()’ on every passenger booking during the simulation.
Preconditions	‘Pricing_rules’ and ‘System_config’ tables must exist. ‘conn’ must be open.
Exceptions	‘pymysql.OperationalError’ on DB failure. If ‘profit_margin’ is absent from ‘System_config’, defaults to ‘0.0.
Postconditions	Returns a three-entry dict (economy, business, first). No writes.

Table 20: load_pricing()

calc_fare()

Type: Function **Return Type:** ‘float’

Parameters:

Name	Purpose	Data Type	Mode
seat_class	Cabin class: economy, business, or first.	‘str’	input
dist_nm	Flight distance in nautical miles	‘float’	input
pricing	Dict of pricing rules keyed by seat class	‘dict’	input

Description	Applies the linear fare formula: ‘(base_price_usd + price_per_nm * dist_nm) * (1 + margin / 100)’ and rounds to two decimal places. This is called once per leg per passenger during ‘simulate_day()’.
Preconditions	‘seat_class()’ must be a key in ‘pricing’ . ‘dist_nm’ must be positive. ‘Pricing’ must be populated by ‘load_pricing()’.
Exceptions	‘KeyError’ if ‘seat_class’ is not in ‘pricing’.
Postconditions	Returns a non-negative float. No side effects. No DB access.

Table 21: calc_fare()

build_timetable_from_static()

Type: Function **Return Type:** ‘list[dict]’

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database connection	‘pymysql.Connection’	input
aircraft_by_tail	Puts each tail number into the database from ‘load_aircraft’	‘dict’	input
distance_map	Pulls the route tuple and puts it into distance in nautical miles form ‘load_distances’	‘dict’	input
distance_id_map	Puts the route tuple into ‘Flight_distances.id’ from ‘load_distances’	‘dict’	input

Description	Iterates ‘STATIC_FLIGHTS’, resolves each entry's tail number to a DB ‘aircraft_id’ and its route to a ‘dist_nm’ / ‘dist_id’ and inserts a row into ‘Sim_timetable’. Entries whose tail or route cannot be resolved (such as Paris to JFK) are silently skipped. Commits once after all inserts. Returns the list of inserted timetable dicts for use as the daily template.
Preconditions	‘Sim_timetable’ table must exist and be empty (cleared by ‘main()’). ‘aircraft_by_tail’ and ‘distance_map’ must be populated. ‘conn’ must be open with ‘autocommit=False’.
Exceptions	‘pymysql.IntegrityError’ on constraint violations. Skipped flights are logged to stdout.
Postconditions	‘Sim_timetable’ contains one row per included flight. Returns a list of dicts. Commits the transaction.

Table 22: build_timetable_from_static()

compute_demand()

Type: Function **Return Type:** ‘dict[tuple, float]’

Parameters:

Name	Purpose	Data Type	Mode
airports	Dict of airport records keyed by IATA code from ‘load_airports’	‘dict’	input

Description	Applies a gravity model to estimate daily passenger demand for every origin to destination pair. It uses the formula: ‘ origin_pop * DAILY_FLY_RATE * MARKET_SHARE * (dest_pop / sum_of_all_other_metros)’
Preconditions	‘Airports’ must be a non-empty dict with ‘metro-pop’ field for each entry.
Exceptions	‘ZeroDivisionError’ if only one airport exists (other_metros_pop = 0). Not possible.
Postconditions	Returns a dict with one entry per valid OD pair. No side effects. No DB access

Table 23: compute_demand()

build_flight_index()

Type: Function **Return Type:** ‘dict[tuple, list[dict]]’

Parameters:

Name	Purpose	Data Type	Mode
instances	All flight instances for the current simulation day	‘list[dict]’	input

Description	Builds an adjacency index from the day's flight instances, excluding cancelled flights and sorting each route's list by ‘dep_min’ ascending. Used by ‘_bfs()’ to quickly look up all non-cancelled flights between any two airports without scanning the full instance list.

Preconditions	‘instances’ must be a list of dicts each with ‘status’, ‘origin’, ‘dest’, and ‘dep_min’ keys. Instance IDs must be assigned before calling.
Exceptions	None
Postconditions	Returns a ‘defaultdict’ keyed by ‘(origin, dest)’ route tuples. Cancelled flights are absent. No writes.

Table 24: build_flight_index()

_bfs()

Type: Function **Return Type:** ‘list[int]’ or [] if none found

Parameters:

Name	Purpose	Data Type	Mode
origin	IATA code of the passenger’s departing airport	‘str’	input
dest	IATA code of the passenger’s destination airport	‘str’	input
earliest_departure	Earliest usable departure time past midnight	‘int’	input
flight_index	Pre-made adjacency index from ‘build_flight_index’	‘dict’	input
seats_remaining	Live seat availability dict.	‘dict’	in/out
min_connection_minutes	Minimum layover time required between two legs	‘int’	input
max_legs	Maximum number of flight segments in schedule per person	‘int’	input

Description	BFS over the day's flight graph using a ‘deque’. Each queue entry holds the following: ‘(legs_used, current_airport, earliest_next_dep, path_taken)’. A visited-state dict keyed by ‘(airport, dep_minute // 90)’ prunes revisits within the same 90-minute time bucket. Hub airports are explored as intermediates before spoke airports to maximise connectivity. Returns the first valid path found or [].
Preconditions	‘origin ≠ dest’. ‘flight_index’ must be built from the same day's instances. ‘seats_remaining’ must be current. ‘earliest_departure ≥ 0’.
Exceptions	None returns ‘[]’ on failure rather than raising.
Postconditions	Returns a list of ‘instance_id’ integers (1--3 elements) on success. ‘seats_remaining’ is not modified by this function. The caller decrements it after booking. No DB writes.

Table 25: _bfs()

route_passenger()

Type: Function **Return Type:** ‘list[int]’

Parameters:

Name	Purpose	Data Type	Mode
origin	IATA code of the passenger’s departing airport	‘str’	input
dest	IATA code of the passenger’s destination airport	‘str’	input
preferred_dep_minute	Random preferred departure time in minutes past midnight	‘int’	input
flight_index	Pre-made adjacency index from ‘build_flight_index	‘dict’	input

	’		
seats_remaining	Live seat availability dict.	‘dict’	in/out

Description	Wraps ‘_bfs’ with three progressive fallback passes to maximize passenger placement. It first tries the standard 30-minute minimum connection from the preferred departure. If that fails, it tries a 15-minute minimum connection. Finally, it tries a 0-minute minimum from 5 AM. Returns the first successful path or ‘[]’ if all three fail.
Preconditions	Same as ‘_bfs()’
Exceptions	None, returns ‘[]’ on failure.
Postconditions	Returns a path list or ‘[]’. No DB writes. ‘Seats_remaining’ is not modified here.

Table 26: route_passengers()

apply_challenge()

Type: Function **Return Type:** ‘str’

Parameters:

Name	Purpose	Data Type	Mode
day	Current simulation day number (1-14)	‘int’	input
instances	Mutable list of today’s flight instance dicts	‘list[dict]’	in/out

Description	Looks up ‘CHALLENGES[day]’ to get the challenge params dict, then iterates all flight instances and applies stochastic mutations: turbulence/congestion/origin delays add random minutes to ‘dep_min’ and ‘arr_min’; jet-stream adjusts ‘dur_min’ and ‘arr_min’ based on bearing; AOG grounds all legs for selected aircraft; eastern cancellations set

	‘status = "cancelled"’ for eligible flights. Normal-ops days leave instances unchanged.
Preconditions	‘day’ must be in \[1, 14\]. Each instance dict must have ‘dep_min’, ‘arr_min’, ‘dur_min’, ‘status’, ‘delay_min’, ‘delay_reason’, ‘origin’, ‘dest’, and ‘aircraft_id’ keys. ‘COORDS’ and ‘EAST_OF_100W’ constants must be defined.
Exceptions	‘KeyError’ if ‘day’ is not in ‘CHALLENGES’ (not possible with valid input).
Postconditions	Some instances may have modified ‘dep_min’/’arr_min’/’dur_min’ or ’status = "cancelled"’. Returns the challenge description string. No DB writes (mutations are in-memory before the INSERT step).

Table 27: apply_challenges()

simulate_day()

Type: Function **Return Type:** ‘dict’

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database connection	‘pymysql.Connection’	input
day	Current simulation day number (1-14)	‘int’	input
timetable	Fixed base schedule (list of dicts from ‘build_timetable_from_static’)	‘list[dict]’	input
airports	Airport records keyed by IATA from ‘load_airports’	‘dict’	input
aircraft	Aircraft records keyed by ‘aircraft_id’	‘dict’	input
pricing	Pricing rules by	‘dict’	input

	seat class from ‘load_pricing’		
demand	OD pair to passengers a day from ‘compute_demand’	‘dict’	input

Description	Orchestrates all five steps for one simulation day: (1) clone timetable into mutable instances, (2) apply the day's operational challenge, (3) insert flight instances into ‘Sim_flight_instances’, (4) generate demand passengers and route each via BFS recording revenue and legs, (5) compute fuel/landing/lease costs and write ‘Sim_daily_report’. Commits once at the end of all five steps.
Preconditions	All reference dicts (‘airports’, ‘aircraft’, ‘pricing’, ‘demand’, ’timetable’) must be fully populated. ‘conn’ must be open with ‘autocommit=False’. ‘Sim_flight_instances’, ‘Sim_passengers’, ‘Sim_passenger_legs’, and ‘Sim_daily_report’ tables must exist.
Exceptions	‘pymysql.IntegrityError’ or ‘pymysql.OperationalError’ on DB write failure. The commit at the end will not execute and the day's data will be inconsistent.
Postconditions	All five Sim tables are updated for this day. Returns a dict of daily totals. Transaction committed.

Table 28: simulate_day()

generate_final_report()

Type: Function **Return Type:** none/

Parameters:

Name	Purpose	Data Type	Mode
conn	Active database connection	‘pymysql.Connection’	input
daily_totals	List of all 14 day result dicts	‘list[dict]’	input

	returned by ‘simulate_day’		

Description	Sums all metric fields across the 14 daily result dicts and inserts a single summary row into ‘Sim_final_report’. Also prints a formatted 14-day summary table to stdout showing total passengers, cancellations, delays, fuel cost, lease, revenue, and net P&L.
Preconditions	‘daily_totals’ must have exactly 14 entries, each with the same keys. ‘Sim_final_report’ table must exist. ‘conn’ must be open.
Exceptions	‘pymysql.OperationalError’ on DB write failure.
Postconditions	One row inserted into ‘Sim_final_report’. Transaction committed. Summary printed to stdout.

Table 29: generate_final_report()

main()

Type: Function **Return Type:** none

Parameters:

Description	Entry point for the simulation script. Connects to the DB, clears all ‘Sim_*tables so the run is always a clean slate, loads all reference data, builds the timetable once from ‘STATIC_FLIGHTS’, runs ‘simulate_day()’ for each of the 14 days, and calls ‘generate_final_report()’. Closes the connection when complete.
Preconditions	MariaDB must be reachable. All reference tables (‘Airports’, ‘Aircraft’, ‘Aircraft_models’, ‘Flight_distances’, ‘Pricing_rules’, ‘System_config’) must be seeded. All ‘Sim_*’ tables must exist.
Exceptions	‘pymysql.OperationalError’ if the DB is unreachable at startup. Any unhandled exception inside ‘simulate_day()’ will abort the run.
Postconditions	All ‘Sim_*’ tables are populated with 14 days of simulation data. DB connection is closed. Final report

	is printed to stdout.

Table 30: main()

panther_cloud_air_shared.js - Shared Browser Utilities

1. Data Components

Name	Purpose	Type
AIRPORTS	Gloabal array of airport objects loaded from '/api/airports' Fields: 'rank', 'iata', 'city', 'name', 'pop', and 'hub'. Overwritten by 'fetchAirports()' every time the page is loaded	JS global 'Array<object>'
CITY	Looks up the city name based on the IATA code by 'fetchAirports()' and is used by 'fillRouteDropdowns()'	JS global 'Object'
HUBS	An array of hub IATA codes	JS global 'Array<string>'
FLEET_DATA	An array of all aircraft and their model records from '/api/aircraft'.	JS global 'Array<object>'
API_BASE	The base URL for all 'fetch()' calls	JS global 'string' (constant at load time)
FLIGHTS	A static array of all 245 scheduled flight objects defined in 'aircraft_schedule.js(). Fields: 'num', 'from', 'to', 'dep', 'arr', 'type', 'tail'	JS global 'Array<object>' (read only)
TOO_CLOSE	Set of '"IATA1-IATA2"' pairs for airports within 150 miles of each other	JS global 'Set<string>' (read only)

	which is defined in ‘aircraft_schedule.js’	
AIRCRAFT	A static array of aircraft objects with daily leg schedules. Also defined in ‘aircraft_schedule.js’	JS global ‘Array<object>’ (read only)
localStorage.pca_cpm	Admin-configured cents-per-nautical-mile pricing rate (goes from 5 - 150)	Browser ‘localStorage’ key
localStorage.pca_margins	Admin-configured cabin margin percentages used by ‘calcMarginMultiplier()’	Browser ‘localStorage’ key
localStorage.pca_theme	User’s theme preference (“light” or “dark”).	Browser ‘localStorage’ key
localStorage.pca__username	Logged-in staff username. Appears in top right corner. Cleared when the browser tab closes.	Browser ‘sessionStorage’ key
localStorage.pca_role	Logged-in user’s role (“admin” or “employee”). Determines what is visible in the nav-bar and accessible.	Browser ‘sessionStorage’ key

showBookingStep	Swaps visible booking panel without a page reload	JS procedure
navigateTo	Universal navigation that handles booking sub-steps via redirect	JS procedure
switchTab	Activates a tab in the Employee or Admin portal	JS procedure
checkAuth	Reads session; redirects to login if absent	JS function ‘object\null’
initNavBar	Populates and shows the nav bar with username, role badge, highlights based on the current	JS procedure

	page in use	
_saveSessionAndRedirect	Saves login session and routes to the role-appropriate portal	JS procedure
logout	Clears session and returns the user to the login page	JS procedure
applyTheme / setTheme	Apply or toggle the light/dark color theme	JS procedures
getCentsPerNm	Returns the current admin-adjusted pricing rate from ‘localStorage’	JS function ‘number’
calcBaseFare	Computes economy base fare from block minutes and pricing rate	JS function ‘number’
areTooClose	Returns true if two IATA codes are in the 150-mile proximity set	JS function ‘boolean’
fillRouteDropdowns	Populates all airport-picker ‘<select>’ elements from the ‘FLIGHTS’ array	JS procedure
showModal / hideModal	Toggle modal overlay visibility	JS procedures
showAircraftPanel / hideAircraftPanel	Open/close the aircraft detail side panel in admin.html	JS procedures
timeToMinutes	Converts “HH:MM” string to integer minutes since midnight	JS function ‘number’
fetchAirports	Async: fetches ‘/api/airports’ and populates ‘AIRPORTS’, ‘CITY’, and ‘HUBS’ globals	Async JS function
fetchAircraft	Async: fetches ‘/api/aircraft’ and populates ‘FLEET_DATA’	Async JS function
fetchPricing	Async: fetches ‘/aip/pricing’ and updates the admin	Async JS function

	pricing status label	

2. Module Signatures:

showBookingStep()

Type: Procedure **Parameters:**

Name	Purpose	Data Type	Mode
stepId	ID suffix of the booking wizard panel to reveal: (“book”, “class”, “confirm”, “done”)	‘string’	input

Description	Hides every element with class ‘pg’ by setting ‘display:none’, then shows the element with id ‘pg-<stepId>’ by setting ‘display:block’. Allows the five-step booking wizard to operate as a single-page app without full reloads, preserving all in-memory booking state.
Preconditions	A DOM element with id ‘pg-<stepId>’ must exist on the current page. Called only from ‘book.html’.
Exceptions	Silently does nothing if the target element is not found.
Postconditions	Exactly one ‘.pg’ panel is visible. All others are hidden.

Table 31: showBookingStep()

navigateTo()

Type: Procedure **Parameters:**

Name	Purpose	Data Type	Mode

pageId	Page or booking-step identifier (such as ‘airports’ or ‘confirm’)	‘string’	input

Description	Universal navigation function. Booking sub-step IDs (‘book’, ‘class’, ‘confirm’, ‘done’) are handled in-place via ‘showBookingStep()’ if already on ‘book.html’; otherwise redirects to ‘book.html’. All other IDs redirect to ‘<pageId>.html’. Also triggers ‘renderConfirmStep()’ or ‘renderDoneStep()’ when navigating to those booking steps to ensure fresh content.
Preconditions	‘FLIGHTS’ global must be defines. For booking sub-steps on ‘book.html’, the corresponding panel elements must exist.
Exceptions	None, unrecognized ID’s redirect to ‘<pageId>.html’
Postconditions	The appropriate page or panel is displayed. The ‘Book Flight’ nav button is highlighted on all booking sub-steps.

Table 32: navigateTo()

switchTab()

Type: Procedure **Parameters:**

Name	Purpose	Data Type	Mode
portalGroup	Portal identifier: ‘emp’ for employee ‘adm’ for admin	‘string’	input
panelId	DOM id of the ‘.pan’ element to reveal	‘string’	input
clickedBtn	The ‘<button>’ element that was clicked	‘HTMLElement’	input

Description	Removes the 'on' class from all tab buttons in the portal's tab bar, adds it to 'clickedBtn', then hides all

	'.pan' panels inside the portal page and shows the one matching 'panelId'. Used in both 'employee.html' and 'admin.html'.
Preconditions	DOM elements with ids ‘emp-tabs’ / ‘adm-tabs’ and ‘pg-employee’ / ‘pg-admin’ must exist. ‘panelId’ must match an existing ‘.pan’ element.
Exceptions	Silently does nothing if any expected element is missing.
Postconditions	Exactly one tab button and one panel are marked active (‘on’ class)

Table 33: switchTab()

checkAuth()

Type: Function **Return Type:** ‘{username: string, role: string}’ or ‘null’

Parameters:

Description	Read ‘pca_uername’ and ‘pca_role’ from ‘sessionStorage’. If ‘pca_username’ is absent or empty, immediately redirects to ‘login.html’ and returns ‘null’. Otherwise returns the session object. Every protected page calls this at the top of its ‘init()’ function.
Preconditions	‘sessionStorage’ must be available (standard in modern browsers)
Exceptions	None, redirect happens synchronously via ‘window.location.href’.
Postconditions	If the user is authenticated, returns ‘{username, role}’ and execution continues. If not, the browser navigates to ‘login.html’ and the calling ‘init()’ receives ‘null’.

Table 34: checkAuth()

initNavBar()

Type: Procedure **Parameters:**

Name	Purpose	Data Type	Mode
auth	Session object form ‘checkAuth()’ (‘{username, role}’) or null	‘object|null’	input

Description	Makes the ‘#nav’ bar visible (it is hidden by default to prevent an unauthenticated flash), writes the username and role badge, shows the Employee nav button for all users, shows the Admin nav button only for Admin-role users, highlights the nav button matching the current page filename, and syncs the theme toggle checkbox.
Preconditions	The ‘#nav’ element and nav button elements must exist in the DOM. Called after ‘checkAuth()' confirms a valid session.
Exceptions	Silently skips steps where target elements are not found. If ‘auth’ is ǹullì, return immediately.
Postconditions	Nav bar is visible. Role-appropriate buttons are shown. Active page button is highlighted.

Table 35: initNavBar()

_saveSessionAndRedirect()

Type: Procedure **Parameters:**

Name	Purpose	Data Type	Mode
username	Authenticated username to store in ‘sessionStorage’	string	input
role	User role controlling the redirect target	string	input

Description	Saves ‘username’ and ‘role’ to ‘sessionStorage’ under the keys ‘pca_username’ and ‘pca_role’, then redirects to ‘admin.html’ for Admin users or ‘employee.html’ for Employee users. Called by ‘login.html’ immediately after a successful ‘/api/login’ response.

Preconditions	‘username’ and ‘role’ must be non-empty strings received from the server. ‘sessionStorage’ must be available.
Exceptions	None
Postconditions	‘sessionStorage’ contains both keys. Browser navigates to the role-appropriate portal page.

Table 36: _saveSessionAndRedirect()

logout()

Type: Procedure **Parameters:**

Description	Removes ‘pca_username’ and ‘pca_role’ from ‘session Storage’, clearing the login session, then redirects to ‘login.html’. Called by the ‘Sign out’ button present in every page’s nav bar.
Preconditions	None. Can be called even when session keys do not exist.
Exceptions	None
Postconditions	‘sessionStorage’ no longer contains ‘pca_username’ or ‘pca_role’. Browser navigates to ‘login.html’.

Table 37: logout()

applyTheme()

Type: Procedure **Parameters:**

Description	Reads ‘pca_theme’ from ‘localStorage’. If the value is “light”, add the “light” class to ‘<body>’ and check the theme toggle checkbox. Otherwise removes it, keeping the default dark theme. Called at the very top of every page’s ‘init()’ to prevent a flash of the wrong theme before content renders.
Preconditions	‘localStorage’ must be available. Called before the page’s main content is shown.
Exceptions	None
Postconditions	‘<body>’ class list reflects the stored theme preference.

	Theme checkbox is synced.

Table 38: applyTheme()

setTheme()

Type: Procedure **Parameters:**

Name	Purpose	Data Type	Mode
isLight	‘true’ to enable light mode and ‘false’ to enable dark mode	boolean	input

Description	Toggles the ‘light’ CSS class on ‘<body>’ to switch the color theme, then persists the choice to ‘localStorage’ under ‘pca_theme’ so it survives page navigation and browser restarts.
Preconditions	‘localStorage’ must be available. Called by the theme toggle checkbox’s ‘onchange’ handler.
Exceptions	None
Postconditions	‘<body>’ class reflects the new theme. ‘localStorage.pcs_theme’ is updated to “light” or “dark”

Table 39: setTheme()

getCentsPerNm()

Type: Function **Return Type:** ‘number’

Parameters:

Description	Reads ‘pca_cpm’ from ‘localStorage’ and returns it as a float. If the stored value is absent or outside the valid range \[5, 150]\, returns the default of 65 cents / nm. Used by ‘calcBaseFare()’ and the admin pricing tab to compute fares consistently across pages.
Preconditions	‘localStorage’ must be available. Valid values are numeric strings in \[5, 150\].

Exceptions	None, values out of range are switched to default values.
Postconditions	Returns a float in \[5, 150]\.

Table 40: getCentsPerNm()

calcBaseFare()

Type: Function **Return Type:** 'number'

Parameters:

Name	Purpose	Data Type	Mode
legs	Array of flight leg objects. Each must have 'dep' and 'arr' time strings in "HH:MM" format	'Array<object>'	input

Description	Sums block minutes across all legs (handling overnight arrivals where 'arr < dep') multiplies by 6 to approximate nautical miles, multiplies by 'getCentsPerNm() / 100' to get dollars, and returns the result floored at \$49. The \$49 floor prevents absurdly cheap short hops.
Preconditions	Each leg object must have 'dep' and 'arr' fields parseable by 'timeToMinutes()'. 'getCentsPerNm() must be callable.
Exceptions	Returns 'NaN' if any time string is malformed.
Postconditions	Returns a positive number >= 49. No side effects. No DB or network access.

Table 41: calcBaseFare()

areTooClose()

Type: Function **Return Type:** 'boolean'

Parameters:

Name	Purpose	Data Type	Mode
airportA	First IATA code	‘string’	input
airportB	Second IATA code	‘string’	input

Description	Returns ‘true’ if the pair appears in the ‘TOO_CLOSE’ set (airports within 150 miles of each other). Checks both orderings (‘A-B’ and ‘B-A’) since ‘TOO_CLOSE’ stores pairs in canonical order. Returns ‘false’ if ‘TOO_CLOSE’ is undefined (graceful degradation).
Preconditions	‘TOO_CLOSE’ should be defined by ‘aircraft_schedule.js’ before this file loads.
Exceptions	None. Returns ‘false’ if ‘TOO_CLOSE’ is undefined
Postconditions	Returns a boolean

Table 42: areTooClose()

fillRouteDropdowns()

Type: Procedure **Parameters:**

Description	Collects every unique IATA code that appears as an origin or destination in ‘FLIGHTS’, deduplicates with a ‘Set’, sorts alphabetically, and builds ‘<option>’ elements appending city names from ‘CITY’. Injects the same option list into the booking, employee, and admin flight-search dropdowns. The schedule filter dropdowns get a leading "All" option. Defaults ‘#b-to’ to the second airport so origin and destination are different on load.
Preconditions	‘FLIGHTS’ global must be defined. ‘CITY’ should be populated before calling for best results. Falls back to codes-only labels if not.
Exceptions	Silently skips dropdown elements not found in the DOM.
Postconditions	All airport picker ‘<select>’ elements are populated.

Table 43: fillRouteDropdowns()

showModal() / hideModal()

Type: Procedures **Parameters:**

Name	Purpose	Data Type	Mode
modalId	DOM id of the modal overlay element to show or hide	‘string’	input

Description	‘showModal’ adds the ‘on’ CSS class to the element with the id ‘modalId’, making the modal backdrop visible and blocking pointer events on the page behind it. ‘hideModal’ removes ‘on’, hiding it. Used for the “Add New Flight” modal in “admin.html”
Preconditions	An element with id ‘modalId’ must exist in the DOM
Exceptions	Silently does nothing if the element is not found.
Postconditions	The modals ‘on’ class is added or removed accordingly.

Table 44: showModal() / hideModal()

showAircraftPanel()

Type: Procedures **Parameters:**

Name	Purpose	Data Type	Mode
tailNumber	Aircraft tail number to display in the panel header Example: N807CA	‘string	input
aircraftType	Model code to display Example: B738	‘string’	input

Description	Opens the slide-in aircraft detail panel (‘#side’) in ‘admin.html’. Populates the panel header with ‘tailNumber’ and ‘aircraftType’, finds the matching aircraft in the static ‘AIRCRAFT’ array, and renders the first four scheduled legs as a timeline (leg 0 = done, leg 1 = current, legs 2-3 = upcoming).
Preconditions	‘#side-tail’, ‘#side-type’, and ‘#side-legs’ DOM elements

	must exist. ‘AIRCRAFT’ global must be defined by ‘aircraft_schedule.js’.
Exceptions	Silently shows an empty timeline if the tail number is not found in ‘AIRCRAFT’
Postconditions	‘#side’ panel is visible ("on" class added). Panel content reflects the specified aircraft's schedule.

Table 45: showAircraftPanel()

hideAircraftPanel()

Type: Procedure **Parameters:**

Description	Removes the “on” CSS class from the ‘#side’ aircraft detail panel element, sliding it off-screen.
Preconditions	‘#side’ element must exist in the DOM.
Exceptions	None
Postconditions	‘#side’ panel is hidden

Table 46: hideAircraftPanel()

timeToMinutes()

Type: Function **Return Type:** ‘number’

Parameters:

Name	Purpose	Data Type	Mode
timeString	Time in “HH:MM” format	‘string	input

Description	Splits the string on “:”, multiplies the hour component by 60 and adds the minute component. Used throughout the code whenever block time, connection gaps, or layover durations are computed.
Preconditions	‘timeString’ must be in “HH:MM” format with valid integer components.

Exceptions	Returns ‘NaN’ if the string is malformed or components are non-numeric.
Postconditions	Returns an integer in \[0, 1439\] for valid inputs.

Table 47: timeToMinutes()

fetchAirports()

Type: Async Function. **Return Type:** Promise<void>

Parameters:

Description	Sends ‘GET /api/airports’, awaits the JSON response, and populates the three global arrays: ‘AIRPORTS’: (iata, city, name, population, hub) ‘CITY’: (IATA to city name map) ‘HUBS’: (filtered HUB IATA codes)
Preconditions	‘API_BASE’ and ‘FLIGHTS’ globals must be defined. Called with ‘await’ during each page’s ‘init()’.
Exceptions	Catches all fetch/parse errors internally. If it logs a warning it uses a fallback and does not re-throw.
Postconditions	‘AIRPORTS’, ‘CITY’, and ‘HUBS’ are populated. ‘fillRouteDropdowns()’ can be called after this.

Table 48: fetchAirports()

fetchAircraft()

Type: Async Function. **Return Type:** Promise<void>

Parameters:

Description	Sends ‘GET /api/aircraft’ and stores the JSON array in the ‘FLEET_DATA’ global for use by the admin Fleet tab. Fails silently on network error, leaving ‘FLEET_DATA’ as an empty array.
Preconditions	‘API_BASE’ global must be defined. Called with ‘await’ during admin page ‘init()’.
Exceptions	Catches all errors internally and logs a warning.

Postconditions	‘FLEET_DATA’ is either populated with fleet records or remains ‘[]’ on failure.

Table 49: fetchAircrafts()

fetchPricing()

Type: Async Function. **Return Type:** Promise<void>

Parameters:

Description	Sends ‘GET /api/pricing’ and writes the count of returned rules into the ‘#pricing-loaded’ DOM element in the admin Pricing tab. On failure writes “DB offline”. This call is informally only live fare calculated using ‘getCentsPerNm()’ from ‘localStorage’.
Preconditions	‘API_BASE’ global must be defined. Called with ‘await’ during the admin page ‘init()’.
Exceptions	Catches all errors internally. Does not re-throw.
Postconditions	‘#pricing-loaded’ text is updated. No global variables are modified.

Table 50: fetchPricing()

book.html -

Data Components

Name	Purpose	Type
FLIGHTS	Static array of all scheduled JS global flight aircraft_schedule.jṡ. The DFS adjacency graph is built from this each time the user searches	JS global Array<object>
TOO_CLOSE	Set of airport pairs	JS global Set<string>

	within 150 miles	
foundItineraries	Module-level array holding up JS to 6 DFS results from the most recent search. Reset on each searchFlights() calls.	JS In-memory
selectedLegs	Array of leg objects for the itinerary the user selected in Step 2. Persists through Steps 3–5.	JS Array<object>
selectedClass	Cabin class chosen in Step 3 . Used by renderConfirmStep() to compute the final price.	JS string
_total	Computed total fare in USD; set by renderConfirmStep() and read by submitBooking()	JS number
adjacencyGraph	Temporary adjacency map built inside searchFlights() from FLIGHTS. Maps origin IATA → Array<flight>. Rebuilt on every search.	JS Object (transient, per-search)
localStorage.pca_cpm	Cents-per-nm pricing rate read via getCentsPerNm() to compute displayed fares	Browser localStorage key (read)
localStorage.pca_margins	Margin multiplier read via getMarginMultiplier() to scale the base fare.	Browser localStorage key
sessionStorage.pca_username	Read by checkAuth() at page init to confirm the user is logged in.	Browser sessionStorage keys

Module Signatures:

searchFlights()

Type: Procedure **Parameters:**

Description	Reads origin, destination, and max-stops from the search form. Validates that origin does not equal destination and they are not in the TOO_CLOSE set. Builds an adjacency graph from FLIGHTS, resets foundItineraries, and calls the inner dfs() starting from the origin
Preconditions	FLIGHTS, TOO_CLOSE, CITY, and areTooClose must be defined. Form elements #b-from, #b-to, #b-stops must exist in the DOM
Exceptions	alert() if origin equals destination. Renders a proximity warning into #results if the pair is too close. No exceptions thrown.
Postconditions	foundItineraries contains 0–6 valid itineraries. #results is updated by displaySearchResults()

Table 51: searchFlights()

dfs()

Type: Inner function of searchFlights **Parameters**: CurrentAirport (string) — IATA code of the airport being expanded; pathSoFar (Array<object>) — ordered array of flight leg objects taken so far in this branch.

Description	Recursive DFS over the adjacency graph. For each outbound flight from currentAirport: skips if the flight number is already in the path (no reuse); skips if the leg is in the TOO_CLOSE set; skips if the connection gap from the previous arrival is outside [30, 360] minutes. If the flight reaches dest, pushes the path to foundItineraries. Otherwise recurses if stop budget allows. Stops when 6 itineraries are found.
Preconditions	foundItineraries, dest, maxStops, and adjacencyGraph must be defined in the enclosing searchFlights scope. areTooClose and timeToMinutes must be callable.
Exceptions	None — invalid branches are skipped via continue.
Postconditions	foundItineraries is populated with valid paths (each an array of leg objects). At most 6 itineraries are added per search.

Table 52: dfs()

displaySearchResults()

Type: Procedure **Parameters:** (none — reads foundItineraries global)

Description	Iterates foundItineraries and builds HTML for up to 6 itinerary result cards, each showing origin/destination airports, departure/arrival times, flight numbers, stop count, layover durations, aircraft type tags, and the economy fare. Injects the HTML into #results. If foundItineraries is empty, shows a "No flights found" message.
Preconditions	foundItineraries must be defined. calcTotalFare, calcLegFare, and timeToMinutes must be callable. #results element must exist in the DOM.
Exceptions	None — empty results show a user-facing message rather than throwing.
Postconditions	#results contains rendered itinerary cards or a no-results message. The first card is pre-selected ("sel" class).

Table 53: displaySearchResults()

selectItinerary(index)

Type: Procedure **Parameters:** index: number — index into foundItineraries of the card the user clicked.

Description	Stores the chosen legs array from foundItineraries[index] into selectedLegs, then calls showBookingStep(3) to advance the wizard to the cabin-class selection screen.
Preconditions	foundItineraries must contain at least index+1 entries. showBookingStep must be defined.
Exceptions	None.

Postconditions	selectedLegs holds the chosen itinerary. The wizard displays Step 3.

Table 54: selectItinerary()

renderCabinStep()

Type: Procedure **Parameters:** (none — reads selectedLegs, pricing from localStorage)

Description	Populates the cabin-class selection panel with economy and first-class fare totals computed from selectedLegs using calcTotalFare(). Renders two selectable cards into the step-3 panel.
Preconditions	selectedLegs must be set. getCentsPerNm and getMarginMultiplier must be callable. The step-3 panel must exist in the DOM.
Exceptions	None.
Postconditions	The step-3 panel shows economy and first-class fare options for the selected itinerary.

Table 55: renderCabinStep()

renderConfirmStep()

Type: Procedure **Parameters:** (none — reads selectedLegs, selectedClass, pricing from localStorage)

Description	Computes the full price breakdown (base fare + taxes + surcharges) for the selected itinerary and cabin class. Populates the Step 4 passenger-detail form fields and the

	fare summary display. Sets the module-level _total variable.
Preconditions	selectedLegs and selectedClass must be set. Pricing helpers must be callable. Step-4 DOM elements must exist.
Exceptions	None.
Postconditions	_total is set to the computed fare. Step 4 form is populated and ready for passenger data entry.

Table 56: renderConfirmStep()

submitBooking()

Type: Async Procedure **Parameters:** (none — reads Step 4 form DOM elements and _total)

Description	Collects passenger data (first_name, last_name, email, dob, seat_class, total_price_usd) from the Step 4 form. POSTs JSON to /api/bookings and awaits the response. On success stores booking_ref in sessionStorage.pca_ref and calls navigateTo("done"). On network failure generates a local fallback reference and still advances to Step 5.
Preconditions	Step 4 form fields must be populated. _total must be set. fetch must be available.
Exceptions	Does not throw — network or API errors are caught and handled with a local fallback reference.
Postconditions	Passenger data is persisted (or fallback reference generated). The wizard advances to Step 5.

Table 57: submitBooking()

renderDoneStep()

Type: Procedure **Parameters:** (none — reads sessionStorage.pca_ref, selectedLegs, selectedClass, _total)

Description	Populates the Step 5 confirmation card with the route summary, cabin class, total price, and booking reference. Also calls GET /api/bookings/<ref> to verify the reference is well-formed and updates the displayed ref with the normalised value returned by the API.
Preconditions	sessionStorage.pca_ref must be set. selectedLegs, selectedClass, and _total must be defined. The step-5 panel must exist in the DOM.
Exceptions	None — API call failure leaves the locally stored ref in place.
Postconditions	The confirmation card is fully populated and visible to the user.

Table 58: RenderDoneStep()

Admin.html - admin dashboard

1. Data Components

Name	Purpose	Type
margins	Profit-margin targets by haul category: sh: short haul, <350 miles mh: medium, 350-1500 mi lh: long, 1500+ miles Intl: JFK to CDG and back	‘object’
simState	14-day simulation state machine: ‘day’, ‘log[]’, ‘cumPax’ (cumulative passengers), and ‘cumCost’ (cumulative operation cost).	‘object’
UNROUTED_DEFAULTS	Hardcoded round-trip legs. Assigned to aircraft that have no legs in the set schedule.	‘object’
_flightsByNum	Lazy index mapping flight number. Built once by ‘_buildFlightIdx()’ and reused by ‘buildPricingAircraftTable()’ and ‘renderDayDetail()’.	‘object /| null’
_detailDayIdx	Index of the currently open day-detail panel. Set by ‘showDayDetail()’, cleared by ‘hideDayDetail()’ and ‘simReset()’.	‘number’
_detailFilter	Hub filter active in the day-detail panel. Set by ‘filterDayDetail()’	‘string’
_cmpFareCache / _cmpFareCacheKey	Cached result of ‘computeCpmAvgFarePerPax()’ and the center per nm value it was computed at. Invalidated	‘number /| null’

	when ‘adjCpmSlider() is called.	
CHALLENGES	Array of 14 day challenges objects imported from ‘aircraft_schedule.js’ that drives the simulation tab.	Array from ‘aircraft_schedule.js’
BASE_FLIGHTS / BASE_PAX / COST_FUEL / COST_LANDING / COST_CDG / COST_MAINT	Fleet-wide daily baseline constants imported from ‘aircraft_schedule.js’ used by ‘simRunDay()’ to compute per-day P&L.	‘number’ constants from ‘aircraft_schedule.js’

getCmpFare	Economy fare for one leg given function block minutes. Authorized to ‘getCentsPerNm()’ from shared.js	Function
computeCpmAvgFareperPax	Fleet-average economy fare per passenger at current cents per nm. Cached per slider value.	Function
adjCpmSlider	Update cents per nm, invalidate cache, reset simulation, refresh all P&L.	Function
resetCpm	Restore default 65 cents per nm and starts full refresh.	Function
computeMarginScale	Revenue multiplier.	Function
adjSlider	Update one margin percentage from slider, persist, refresh P&L.	Function
updatePricingImpact	Cascade refresh of pricing table, route table, simulation log, open day detail.	Function (procedure)
saveMargins	Visual “Saved!” button feedback.	Function (procedure)
resetMargins	Restore default margin	Function (procedure)

	values, persist, and refresh.	
applyAutoRoutes	Assign default legs to aircraft that have none and inject into FLIGHTS.	Function (procedure)
builtFleet	Redner Fleet tab per-aircraft performance table.	Function (procedure)
buildFleetModelStatus	Renderer-model KPI tiles: count, seats, daily lease, in the Fleet tab header.	Function (procedure)
buildRouteTable	Aggregate top-20 routes by passenger volume and render with live P&L.	Function (procedure)
buildPricinAircraftTable	Per-aircraft daily P&L in Pricing tab, sorted by descending P&L.	Function (procedure)
addMinsToTime	Shift “HH:MM” string forward by N minutes with 24 h wrap.	Utility function
mkRng	Determinstic xorshirft32 PRNG seeded per day and returns a closure.	Function
simRunDay	Simulate one operational day. Draws from RNG, computes active flights/pax/revenure/costs and pushes to log.	Function (procedure)
simRunAll	Loop ‘simRunDay()’ until all 14 days are complete.	Function (procedure)
simReset	Clears ‘simState, hides detail panel, and rerenders the empty simulation UI.	Function (procedure)
fmtM	Format a dollar amount based on over or under 1 million dollars.	Utility Function
fmtK	Format a passenger count based on size	Utility Function
refreshSimUI	Re-renders the	Function (procedure)

	simulation log table. Shows a cumulative summary.	
buildSparkbar	Renders a mini daily P&L bar chart: green for profit and red for loss	Function (procedure)
_buildFlightIdx	Lazily builds '_flightsByNum' index from 'FLIGHTS'.	Utility Function
showDayDetail	Opens the per-aircraft detail panel for a completed simulation day.	Function (procedure)
hideDayDetail	Closes and resets the day-detail panel.	Function (procedure)
filterDayDetail	Updates the hub filter in the open day-detail pane and re-renders.	Function (procedure)
renderDayDetail	Re-renders the per-aircraft table for the open day.	Function (procedure)
init	Async startup that authorized from the API who should be on that page. Also restores the slider.	Async function

2. Module Signatures

getCpmFare()

Type: Function **Return Type:** 'number'

Parameters:

Name	Purpose	Data Type	Mode
bm	Block minutes for the flight leg	number	in

Description	Converts a leg's block-minute duration to an economy

	fare using the current cents per nm slider value from ‘getCentsPerNm()’ (shared.js). Formula: max($49, round(bm × 6 × cpm / 100)) 6 nm/min is the approximate domestic ground-speed proxy (\~450 kt).
Preconditions	‘getCentsPerNm()’ must return a valid positive number (stored in ‘localStorage.pca_cpm’, defaulting to 65).
Exceptions	None, ‘Math.max’ guarantees a non-negative number.
Postconditions	Returns the per-seat economy fare for that leg. Does not modify any state.

Table 58: getCpmFare()

computeCpmAvgFareperPax()

Type: Function **Return Type:** ‘number’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Block minutes for the flight leg	Reads ‘FLIGHTS’ global and current ‘getCentsPerNm()’ value.	

Description	Computes the fleet-wide average economy fare per passenger at the current cents per nm rate by summing ‘getCpmFare(bm) * pax’ across all flights and dividing by total passengers. Result is cached in ‘_cpmFareCache’ / ’_cpmFareCacheKey’; cache is invalidated by ‘adjCpmSlider()’. Used by ‘simRunDay()’ to set per-day base revenue so the simulation reflects the current slider setting.
Preconditions	‘FLIGHTS’ must be populated by ‘fetchAircraft()’ and/or ‘applyAutoRoutes()’. ‘getCentsPerNm()’ must returns valid positive number.
Exceptions	None.
Postconditions	Returns the average fare. Writes ‘_cpmFareCache’ and ‘_cpmFareCacheKey’ on cache miss.

Table 59: computeCpmAvgFareperPax()

adjCpmSlider()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
v	Raw slider value from the range input.	‘string | number’	in

Description	Clamps the input to \[5, 150\], saves to ‘localStorage.pca_cpm’, invalidates the fare cache, updates the label and sample-fare display, resets ‘simState’ to day 0 (because stored ‘baseRev’ values would be inconsistent with the new rate), hides the detail panel, and calls ‘updatePricingImpact()’ + ‘refreshSimUI()’.
Preconditions	Cents per nm slider DOM element and label elements must exist.
Exceptions	None. Clamping ensures the value is always in range.
Postconditions	‘localStorage.pca_cpm’ updated and ‘simState’ reset to {day:0, log:[], cumPax:0, cumCost:0}; all P&L displays refreshed.

Table 60: adjCpmSlider()

resetCpm()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Resets slider DOM value and delegates to ‘adjCpmSlider(65)’		in

Description	Restores the cents per nm slider to the default value of

	65 and triggers the full refresh via ‘adjCpmSlider(65)’
Preconditions	Slider DOM element ‘#cpm-slider’ must exist.
Exceptions	None
Postconditions	Cents per nm is 65, the simulation is reset, and all P&L displays are refreshed.

Table 61: resetCpm()

computeMarginScale()

Type: Function **Return Type:** ‘number’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘margins’ global		

Description	Returns the revenue multiplier for the current margin settings relative to the 24.5% baseline (average of default sh=18%, mh=22%, lh=26%, intl=32%). Formula: (1 + curr/100) / (1 + 24.5/100) A value \>1.0 means margins are set above baseline and \<1.0 means below. Applied on top of the cents per nm base fare in route, pricing, and simulation displays.
Preconditions	‘Margins’ object must have ‘sh’, ‘mh’, ‘lh’, ‘and ‘intl’ properties set to numbers in \[1, 99]\.
Exceptions	None.
Postconditions	Read-only and does not modify any state.

Table 62: computeMarginScale()

adjSlider()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode

k	Margin key is ‘sh’, ‘mh’, ‘lh’, or ‘intl’	‘string’	in
v	Raw slider value from the range input	‘string | number’	in

Description	Clamps ‘v’ to \[1, 99\], assigns to ‘margins[k]’, updates the corresponding label element, persists the full ‘margins’ object to ‘localStorage.pca_margins’ and calls ‘updatePricingImpact()’.
Preconditions	A label element with id ‘k + ‘-v’’ must exist in the DOM.
Exceptions	None. The clamping ensures value is always in range.
Postconditions	‘Margins[k]’ updated, ‘localStorage.pca_margins’ updated, and all P&L displays are refreshed.

Table 63: adjSlider()

updatePricingImpact()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘_detailDayIdx’ to decide whether to re-render day detail		

Description	Cascade refresh triggered on every slider change. Calls ‘buildPricingAircraftTable()’, ‘buildRouteTable()’, ‘refreshSimUI()’, and (if ‘_detailDayIdx >= 0’) ‘renderDayDetail()’. Ensures all P&L-sensitive UI stays in sync with the current margin and cents per nm settings.
Preconditions	All dependent build/render functions must be defined.
Exceptions	None
Postconditions	Pricing tab, Route tab, simulation log, and day-detail panel (if open) all reflect current slider state.

Table 64: updatePricingImpact()

saveMargins()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘even.target’ to get the button element.		

Description	Visual feedback. It changes the button text to “Saved!” and background to green for 1.5 seconds and restores. No data is sent to d the backend and persistence is handled in ‘adjSlider()’
Preconditions	Must be called from a button’s ‘onclick’ handler so ‘event.target’ resolves to the button.
Exceptions	None.
Postconditions	No persistent state change and purely cosmetic.

Table 65: saveMargins()

resetMargins()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Restores all four margin sliders to their default values.		

Description	Resets ‘margins’ to ‘{sh:18, mh:22, lh:26, intl:32}’, updates all slider DOM element and labels, persists to ‘localStorage.pca_margins’, and calls ‘updatePricingImpact()’
Preconditions	Slider (‘k + ‘-slider’’) and label (‘k + ‘-v’’) DOM elements must exist for each key.

Exceptions	None
Postconditions	‘Margins’ and ‘localStorage.pc_margins’ both reflect the default values and all P&L displays are refreshed.

Table 66: resetMargins()

applyAutoRoutes()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘AIRCRAFT’, ‘FLIGHTS’, ‘localStorage.pca_auto_routes’, and ‘UNROUTED_DEFAULTS’		

Description	Iterates ‘AIRCRAFT’. For any aircraft with no legs, it assigns routes based on priority order from ‘localStorage.pca_auto_routes’, ‘UNROUTED_DEFAULTS’, or a generic out-and-back route. Injects the new legs into ‘FLIGHTS’ so routes, pricing, and search data remain consistent. Saves new assignments back to ‘localStorage.pca_auto_routes’.
Preconditions	‘AIRCRAFT’ and ‘FLIGHTS’ must already be populated by ‘fetchAircraft()’
Exceptions	JSON parse errors from ‘local’Storage’ are silently caught with empty-object fallback.
Postconditions	Every aircraft in ‘AIRCRAFT’ has at least one leg. ‘FLIGHTS’ contain entries for all auto assigned legs. ‘localStorage.pca_auto_routes’ updated if any new assignments were made.

Table 67: applyAutoRoutes()

buildFleet()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘AIRCRAFT’ and ‘FLIGHTS’ globals. Writes to ‘#fleet-body’.		

Description	Aggregates passengers and seats per tail number from ‘FLIGHTS’, estimates block hours as ‘legs × 2.1 h’, and assigns a cyclic maintenance-hours sample from a hardcoded array. Aircraft with maintenance hours \>185 receive a "Maint." badge. Builds an HTML table row per aircraft and injects it into ‘#fleet-body’. Each row includes a "View" button that calls ‘showAircraftPanel()’ from shared.js.
Preconditions	‘AIRCRAFT’ and ‘FLIGHTS’ must be populated. DOM element ‘#fleet-body’ must exist.
Exceptions	None
Postconditions	‘#fleet-body’ contains one row per aircraft with current leg counts, load factors, and operational status.

Table 68: buildFleet()

buildFleetModelStats()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘FLEET_DATA’. Writes to ‘#fleet-model-stats’.		

Description	Groups ‘FLEET-DATA’ entries by ‘model_name’ and accumulates count, and reads seat configuration and monthly lease from the first matching entry. Renders one KPI tile per model showing count, seat count, and daily lease cost (‘monthly_lease_usd / 30’).
Preconditions	‘FLEET-DATA’ must be populated. DOM element ‘#fleet-model-stats’ must exist.

Exceptions	Returns early without rendering if ‘FLEET_DATA’ is empty.
Postconditions	‘#fleet-model-stats’ contains one tile per aircraft model.

Table 69: buildFleetModelStatus()

buildRouteTable()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘FLIGHTS’ and calls ‘getCpmFare()’, ‘computeMarginScale()’, and writes to ‘#route-body’.		

Description	Sorts all flights by origin--destination pair, computing total passengers, seats, and block minutes. Sorts routes by descending passenger count and renders the top 20. For each route: Average fare = ‘getCpmFare(total_bm / n)’ Revenue = ‘avg_fare × pax × computeMarginScale()’ Fuel = ‘(total_bm / 60) × 800 × $6.19/gal’ Fees = ‘n * $2,000’ Routes that start or end at a hub receive a “Hub” badge.
Preconditions	‘FLIGHTS’ must be populated and DOM element ‘#route-body’ must exist.
Exceptions	None
Postconditions	‘#route-body’ contain up to 20 rows, sorted by passenger volume, with live revenue/P&L reflecting current slider values.

Table 70: buildRouteTables()

buildPricingAircraftTable()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘AIRCRAFT’, ‘_flightsByNum’, ‘computerMarginScale()’ and ‘COST_LEASE’. Writes to ‘#pricing-aircraft-body’ and summary elements.		

Description	Calls ‘buildFlightIdx()’ to ensure ‘_flightsByNum’ is ready. For every aircraft, sums revenue (‘getCpmFare(bm) * flt.pax * scale’) and costs (fuel + landing fees + per_aircraft share of ‘COST_LEASE’) across all legs. Sorts rows by descending P&L and renders the tables. Also updates the summary elements ‘#pricing-total-rev’, ‘#pricing-total-cost’, and ‘#pricing-total-pl’.
Preconditions	‘AIRCRAFT’, ‘FLIGHTS’, and ‘FLEET_DATA’ must be populated and relevant DOM elements must exist.
Exceptions	None.
Postconditions	‘#pricing-aircraft-body’ and summary elements reflect current cents per nm and margins settings.

Table 71: buildPricingAircraftTable()

addMinsToTime()

Type: Function **Return Type:** ‘string’

Parameters:

Name	Purpose	Data Type	Mode
t	Base time string in "HH:MM" format	‘string’	in
m	Minutes to add (may be 0)	‘number’	in

Description	Parses t into total minutes, adds m, wraps at

	24 h (1440 min), and formats back to "HH:MM" with leading-zero padding. Used by renderDayDetail() to shift flight departure and arrival times forward by accumulated delay minutes.
Preconditions	t must be a valid "HH:MM" string.
Exceptions	None, returns t unchanged if m is falsy.
Postconditions	No state changes.

Table 72: addMinsToTime()

mkRng()

Type: Function **Return Type:** ‘number’ in\[0,1)

Parameters:

Name	Purpose	Data Type	Mode
seed	Initial PRNG seed	‘number’	in

Description	Returns a stateful xorshift32 random-number generator seeded with seed. Each call to the returned closure executes. Both simRunDay() and renderDayDetail() use the same deterministic seed formula (dayIdx * constant + offset) so the same day always produces identical results regardless of when it is rendered.
Preconditions	seed must be a non-negative integer; a seed of 0 is forced to 1 to avoid the zero-state trap.
Exceptions	None
Postconditions	Each invocation of the returned closure advances the internal state by one step and returns a pseudo-random float. No global state is modified.

Table 73: mkRng()

simRunDay()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘simState.day’, ‘CHALLENGES’, baseline constants. Writes to ‘simState.log’		

Description	Simulates one operational day. Uses ‘CHALLENGES[day]’ for the challenge parameters and a deterministic PRNG seeded by ‘day × 2654435761 + 1’. Computes: cancelled and active flights, load-factor-adjusted passengers, CPM-based ‘baseRev’ (stored pre-margin so ‘refreshSimUI()’ can rescale live), and individual cost components (fuel scaled by ‘weather_factor’, fixed lease, landing fees proportional to active flights, CDG international fee, maintenance). Increments ‘simState.day’, pushes the entry to ‘simState.log’, and calls ‘refreshSimUI()’.
Preconditions	‘simState.day’ must be \< 14; ‘CHALLENGES’, ‘BASE_FLIGHTS’, ‘BASE_PAX’, and cost constants must be populated from ‘aircraft_schedule.js’. ‘computeCpmAvgFarePerPax()’ must return a valid number.
Exceptions	Alerts and returns early if all 14 days are already complete.
Postconditions	‘simState.day’ incremented by 1. One entry added to ‘simState.log. ‘simState.cumPax’ and ‘simState.cumCost’ updated. Simulation UI re-rendered.

Table 74: simRunDay ()

simRunAll()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Loops ‘simRunDay()’ until ‘simState.day =		

	14’		

Description	Calls simRunDay() in a while loop until all 14 days have been simulated. Alerts and returns early if already complete.
Preconditions	Same as simRunDay()
Exceptions	Alerts if already at day 14
Postconditions	simState.day = 14. simState.log has 14 entries. UI fully rendered.

Table 75: simRunAll()

simReset()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Resets simState and hides the detail panel.		

Description	Reinitialized simState to {day:0, log:[], cumPax:0, cumCost:0}, calls hideDayDetail(), and calls refreshSimUI() to render the empty simulation UI.
Preconditions	None
Exceptions	None
Postconditions	Simulation is at day 0 with no log entries and no open detail panel.

Table 76: simReset()

fmtM()

Type: Function **Return Type:** ‘string’

Parameters:

Name	Purpose	Data Type	Mode
n	Dollar amount (positive or negative)	‘number’	in

Description	Formats a dollar amount compactly. Values greater or equal to $1,000,000 become “X.XXM” and maller are valued with a k.
Preconditions	None
Exceptions	None
Postconditions	No state changes

Table 77: fmtM()

fmtK()

Type: Function **Return Type:** ‘string’

Parameters:

Name	Purpose	Data Type	Mode
n	Passenger count	number	in

Description	Formats a passenger count. Greater than 1000 are shown with a “k”.
Preconditions	None
Exceptions	None
Postconditions	No state changes

Table 78: fmtK()

refreshSimUI()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads simState, CHALLENGES, calls computeMargineScale(), writes to #sim-log, #sim-summary, #sim-prog, #sim-day-label #sim-plbars-mini		

Description	Re-renders the entire simulation panel. First it applies computeMarginScale() to each stored baseRev to produce a live-scaled display copy of the log --- this means P&L figures update instantly when margin sliders change without re-running the simulation. Builds the 14-row log table (completed rows are clickable, the next-up row shows the challenge name, pending rows are dimmed), updates the cumulative summary block, and calls buildSparkbar().
Preconditions	simState and CHALLENGES must be populated and DOM elements must exist.
Exceptions	None
Postconditions	All simulation DOM elements reflect current simState and live margin settings.

Table 79: refreshSimUI()

buildSparkbar()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
elId	ID of the container DOM element	‘string’	in
log	Array of scaled log entries with .pl field	‘array’	in

total	Total number of days (14)	‘number’	in

Description	Renders a mini bar chart where each column represents one day. Profit days show a green bar growing upward. Loss days show a red bar growing downward. Pending (not-yet-run) days show a dashed placeholder. Bar heights are proportional to the maximum absolute P&L across all completed days.
Preconditions	DOM element with elId must exist
Exceptions	Returns without rendering if the element is not found. Guards maxAbs = 0 to avoid division by 0.
Postconditions	The container element is replaced with the spark bar HTML.

Table 80: buildSparkbar()

_buildFlightIdx()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads ‘FLIGHTS’. Writes to _flightsByNum		in

Description	Lazily builds the _flightsByNum index by iterating FLIGHTS once the keying each entry by f.num. Subsequent calls are no-ops if the index already exists. Called by buildPricingAircraftTable() and showDayDetail()
Preconditions	FLIGHTS must be populated
Exceptions	None
Postconditions	_flightsByNum is populated with all flight records keyed by flight number

Table 81: _buildFlightIndx()

showDayDetail()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
dayIdx	Zero-based index of the completed simulation day to display.	‘number’	in

Description	Calls _buildFlightIdx() to ensure the flight index is ready, verifies simState.log[dayIdx] exists, sets _detailDayIdx and resets _detailFilter to “All”, resets all hub-filter button styles, calls renderDayDetail(), then shows the smooth-scrools to #sim-day-detail.
Preconditions	simState.log[dayIdx] must exist (day must be completed). #sim-day-detail must exist in the DOM.
Exceptions	Returns without action if the log entry does not exist.
Postconditions	#sim-day-detail is visible and populated with the selected day's per-aircraft detail.

Table 82: showDayDetail()

hideDayDetail()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Hides #sim-day-detail and resets _detailDayIdx		

Description	Sets #sim-day-detail to display: none and resets _detailDayIdx to -1. Called by simReset() and

	adjCpmSlider().
Preconditions	DOM element #sim-day-detail must exist.
Exceptions	None
Postconditions	Detail panel is hidden. _detailDayIdx = -1.

Table 83: hideDayDetail()

filterDayDetail()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
hub	Filter value of hubs	‘string’	in

Description	Updates _detailFilter to ‘hub’, updates the visual active state on all hub filter buttons, and calls renderDayDetail() to re-render the detail table with the new filter applied.
Preconditions	_detailDayIdx must be >= 0. Button elements #ddf-ALL through #ddf-INTL must exist.
Exceptions	None
Postconditions	_detailFilter updated. Detail panel re-rendered showing only aircraft for the selected hub.

Table 84: filterDayDetail()

renderDayDetail()

Type: Procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads _detailDayIdx,		

	_detailFilter, simState.log, CHALLENGES, AIRCRAFT, _flightsByNum. Writes to #day-detail-title, #day-detail-challenge #day-detail-summary #day-detail-content		

Description	Two-pass per-aircraft cascade renderer. Pass 1 (all aircraft): for each aircraft, iterates its legs in schedule order. On each leg, consumes exactly 3 RNG values (for sequence stability). If r1 < cancel_pct, the flight is cancelled and the aircraft is marked “grounded” (all subsequent legs on that aircraft become "Grounded"). If r2 < delay_chance, a 15--75-minute delay is added to accDelay, shifting every subsequent departure and arrival. Computes per-leg revenue and fuel. Accumulates per-aircraft totals. Pass 2 (filtered): renders only aircraft whose hub matches _detailFilter. Outputs leg rows with adjusted dep/arr times and status badges, plus a per-aircraft subtotal row. Updates the header with live P&L using baseRev * computeMarginScale().
Preconditions	_detailDayIdx >= 0 simState.log[_detailDayIdx] must exist _buildFlightIdx() must have been called
Exceptions	Returns without rendering if _detailDayIdx < 0
Postconditions	#day-detail-content contains the per-aircraft cascade table. aHeader shows live P&L.

Table 85: renderDayDetail()

init()

Type: Async procedure **Return Type:** ‘void’

Parameters:

Name	Purpose	Data Type	Mode
(none)	Reads sessionStorage for auth and calls multiple shared.js functions		

Description	Page startup entry point. Calls applyTheme() and checkAuth(); redirects to employee.html immediately if the authenticated role is not “Admin”. Then calls initNavBar(), fires fetchAirports(), fetchAircraft(), and fetchPricing() in parallel with Promise.all(), restores all slider positions from localStorage, calls applyAutoRoutes(), and builds all four tab UIs: fillRouteDropdowns() buildFleet(), buildFleetModelStats(), buildRouteTable(), refreshSimUI(), updatePricingImpact().
Preconditions	sessionStorage must contain a valid login token with role = “Admin”. The Flask server must be running and all three API endpoints must be reachable
Exceptions	If checkAuth() returns false, execution stops. If fetchAirports/fetchAircraft/fetchPricing rejects, the Promise.all will reject and subsequent UI setup will not run.
Postconditions	All four tabs are fully populated with live data. Simulation UI is in the empty/pending state. All slider positions match persisted localStorage values.

Table 86: init()

Work Breakdown

Gracie Mitchell: I went through each of my project files systematically to identify every function that contained meaningful logic worth documenting. I read through simulate.py, book.html, employee.html, app.py, and panther_cloud_air_shared.jsand noted which functions were doing real algorithmic work versus simple utility tasks like rendering HTML or reading a dropdown value. For each algorithm I

wrote a plain English description explaining what the function does, why it was needed, and any notable design decisions. I wrote these descriptions so the reader understands the purpose before seeing the implementation.

Reid Holloway: For this deliverable, I went through book.html, admin.html, panther_cloud_air_shared.js, simulation.py, and db.py and summarized every function and its importance.
I listed different descriptions, preconditions, exceptions, and postconditions.

Graham McCabe: For this deliverable, I went through the SQL files and app.py files and converted the functions into tables that break them down. I also created the final architecture diagram shown in the architecture section, breaking it down into separate layers frontend to backend.